# PyPSA-IEI Documentation

**Integrated Energy Infrastructure Model**

*Open Energy Transition*

*None*

# Table of contents

# 1. PyPSA-IEI Documentation

**PyPSA-IEI** is an energy-system model based on the open-source PyPSA-Eur framework (release 0.10), extended to meet the needs of the study:

> "*Integrated Infrastructure Planning and 2050 Climate Neutrality: Deriving Future-Proof European Energy Infrastructures*" — Fraunhofer IEG, Fraunhofer ISI and d-fine (November 2025)
>
> Download report (PDF)

## 1.1 About the Model

PyPSA-IEI performs **myopic pathway optimisation** of the European energy system across multiple planning horizons between 2020 and 2050 in 5 year increments. It co-optimizes the electricity, hydrogen, gas, heat, and industry sectors within an integrated framework to identify the least-cost configuration of the overall energy system.

Key extensions on top PyPSA-Eur v0.10 include:

- TYNDP-based electricity and gas network enforcement
- Hydrogen network (European Hydrogen Backbone + ENTSO-G)
- Energy import modelling (hydrogen, syngas, synfuel)
- TransHyDe-based industrial demand
- Custom cost assumptions and powerplant lifetimes

## 1.2 Quick Links

- Installation — set up the environment and input data
- Scenarios — overview of the four main policy scenarios (CE, CN, SE, SN)
- Model Functionality — detailed description of model extensions

## 1.3 Contributors

**d-fine GmbH** — Robert Beestermöller, Caroline Blocher, Lukas Czygan, Linus Erdmann, Felix Greven, Paula Hartnagel, Julian Hohmann, Paul Kock, Julius Meyer, Ari Pankiewicz, Anna Thünen

**Fraunhofer IEG** — Benjamin Pfluger, Mario Ragwitz, Caspar Schauß

**Fraunhofer ISI** — Khaled Al-Dabbas, Sirin Alibas, Wolfgang Eichhammer, Tobias Fleiter

For inquiries: wolfgang.eichhammer@isi.fraunhofer.de or info@d-fine.com

# 2. Installation

## 2.1 Requirements

- Git — for cloning the repository (**Windows:** Git for Windows is recommended, which also includes **Git Bash**)
- Conda (Miniconda or Anaconda) — for environment management
- Gurobi — a commercial solver licence is required to run the model locally. If you do not have a Gurobi licence, use the OETC Cloud Solver instead.

## 2.2 1. Clone the Repository

```
git clone https://github.com/open-energy-transition/PyPSA-IEI.git
cd PyPSA-IEI
```

**Working directory**

All Snakemake commands in this guide must be run from inside the `PyPSA-IEI/` directory. The `cd PyPSA-IEI` above puts you there after cloning. If you open a new terminal, run:

```
cd /path/to/PyPSA-IEI
```

## 2.3 2. Download Input Data

This repository only contains input data explicitly developed for this study. The following files must be copied from PyPSA-Eur v0.10.0 (February 19, 2024) into the `data/` folder of this repository:

```
pypsa-eur/                          ← source repository
└── data/
    ├── GDP_PPP_30arcsec_v3_mapped_default.csv
    ├── switzerland-new_format-all_years.csv
    └── urban_percent.csv
```

Additional data will be automatically retrieved by Snakemake rules during the first run.

**Note**

The data used to generate the study results were downloaded on **June 27th 2024**.

## 2.4 3. Set Up the Environment

Create and activate the conda environment using the file matching your operating system:

**Windows** **Linux**

```
conda env create -f envs/environment_pypsa_iei.yaml
conda activate pypsa-iei

conda env create -f envs/environment_pypsa_iei_linux.yaml
conda activate pypsa-iei
```

For detailed instructions on setting up PyPSA-Eur, refer to the official documentation.

---

## 2.5 4. Run a Scenario

Results for the study were generated across four main scenarios and four sensitivities. Choose the track that matches your situation:

---

### 2.5.1 Track 1 — Pipeline Test (Low-End Machine)

This track is designed for running the model on a low-end or Windows machine, or for anyone who wants to verify the full pipeline works end-to-end before committing to a full run. The goal is to test the workflow, not to reproduce study results. Indicative requirements per scenario:

| Configuration | RAM | Cores | Time |
|---|---|---|---|
| `20SEG-T-H-B-I-A` (recommended reduced-resolution test) | >16 GB | 4 | ~1.5-2 hours |
| `20SEG` (electricity-only fallback, limited downstream compatibility in post-analysis) | >16 GB | 4 | ~1.5-2 hours |

**No local solver?**

If you do not have a Gurobi licence or prefer to offload solving, see the OETC Cloud Solver section below.

**Which config to use for testing?**

Use `config.SE.yaml` for pipeline testing. It has fewer custom constraints and works correctly even when sector suffixes (`T`, `H`, `B`, `I`, `A`) are omitted. `config.SN.yaml` enables self-sufficiency constraints that require industry sector components (e.g. Fischer-Tropsch links) to be present — running it without the `I` suffix will cause an error.

#### Step 1 — Reduce the temporal resolution

Edit `sector_opts` in your scenario config to use fewer time segments while keeping the sector suffixes enabled:

```
# config/scenarios/config.SE.yaml
scenario:
  sector_opts:
  - 20SEG-T-H-B-I-A   # reduced from 2190SEG — much faster, less memory
```

This is the recommended Track 1 setup. It keeps the workflow closer to the full study configuration and avoids downstream issues in summary generation and post-analysis scripts that can appear with electricity-only runs.

If you only want the lightest possible smoke test, you can still omit the sector suffixes entirely:

```
scenario:
    sector_opts:
    - 20SEG
```

However, this electricity-only variant should be treated as a minimal pipeline check only. Later postprocessing and analysis steps are more likely to **fail with an electricity-only model**.

> **Warning**
>
> ```
> Reduced-segment results are **not comparable** to the study. Electricity-only
> runs are even more limited and are mainly useful for quick smoke tests.
> ```

#### Step 2 — Dry run (recommended)

Verify that Snakemake can resolve all rules and inputs before executing anything:

```
~/PyPSA-IEI$ snakemake -call all --configfile config/scenarios/config.SE.yaml --dry-run
```

#### Step 3 — Run the full workflow

Use the `all` target so the full pipeline runs and the summary outputs needed for later analysis are generated. Start with as many cores as your machine can handle:

```
~/PyPSA-IEI$ snakemake -call all --configfile config/scenarios/config.SE.yaml
```

If you are on Windows, you can still start with the `all` target as shown above. If the workflow later becomes unstable when it reaches `solve_sector_networks`, see Troubleshooting and resume the same target with `--cores 1`.

Once the run completes, see Verifying Success to confirm the output files are in place. If you run into issues, see the Troubleshooting section.

---

## 2.5.2 Track 2 — Full Study Run

This track reproduces the study results. It requires a machine with sufficient memory (approximately **250 GB of RAM**) and compute time (approximately **2.5 days** per scenario).

**Step 1 — Dry run (recommended)**

Before launching the full run, verify that Snakemake can resolve all rules and inputs without executing anything:

```
~/PyPSA-IEI$ snakemake -call all --configfile config/scenarios/config.SN.yaml --dry-run
```

This prints the list of jobs that would be executed. If any rule or input file is missing, the error appears here — before committing hours of compute time.

**Step 2 — Run all steps with the full configuration:**

```
~/PyPSA-IEI$ snakemake -call all --configfile config/scenarios/config.SN.yaml
```

> **Note**
>
> The study uses **2190 time segments** ( `2190SEG` ) with all sector suffixes ( `-T-H-B-I-A` ). Do not modify `sector_opts` if you want results comparable to the publication.

> **Warning**
>
> On Windows, the workflow may become unstable once it reaches `solve_sector_networks`. Start with as many cores as you want, but if the solve stage fails, resume with `--cores 1` as described in Troubleshooting.

## 2.5.3 Verifying Success

Snakemake prints the following when all jobs complete without errors:

```
X of X steps (100%) done
```

For each planning horizon (2020–2050), a solved network file should appear under `results/<run_name>/postnetworks/`. The exact filename depends on your track:

| Track | Example postnetwork file |
|---|---|
| Track 1 — `20SEG-T-H-B-I-A` | `elec_s_62_lv99__20SEG-T-H-B-I-A_2020.nc` |
| Track 2 — `2190SEG-T-H-B-I-A` | `elec_s_62_lv99__2190SEG-T-H-B-I-A_2050.nc` |

If any files are missing, check the corresponding log file under `logs/` for the failing rule.

For further details on running PyPSA-Eur-based models, refer to the open-source documentation.

If you run into issues, see the Troubleshooting section.

## 2.5.4 OETC Cloud Solver (Optional — Skip Local Solving)

If you prefer not to run the solver locally — whether because of hardware limitations or to save time — the solving step can be offloaded to the Open Energy Transition Cloud (OETC). This works with both Track 1 and Track 2. Network preparation always runs locally; only the solve step is offloaded.

**Step 1 — Enable OETC in your scenario config:**

The `oetc` block is already present in `config/config.agora.yaml` with OETC disabled by default. To enable it, add the following override to your scenario config:

```yaml
# config/scenarios/config.SE.yaml  (or any other scenario file)
solving:
  oetc:
    enabled: true
```

> **Note**
>
> Set `enabled: false` or omit the override entirely to fall back to the local solver.

**Step 2 — Set your OETC credentials as environment variables:**

Linux Windows (Git Bash) Windows (PowerShell)

Add to your `~/.bashrc` (or `~/.bash_profile`) for a persistent setup:

```
nano ~/.bashrc   # or: vi ~/.bashrc
```

Append these lines:

```
export OETC_EMAIL="your@email.com"
export OETC_PASSWORD="yourpassword"
```

Then reload and verify:

```
source ~/.bashrc
echo $OETC_EMAIL
```

Add to your `~/.bashrc`:

```
nano ~/.bashrc   # or: vi ~/.bashrc
```

Append these lines:

```
export OETC_EMAIL="your@email.com"
export OETC_PASSWORD="yourpassword"
```

Then reload and verify:

```
source ~/.bashrc
echo $OETC_EMAIL
```

For the current session only:

```
$env:OETC_EMAIL = "your@email.com"
$env:OETC_PASSWORD = "yourpassword"
```

For a persistent setup (user-level):

```
[System.Environment]::SetEnvironmentVariable("OETC_EMAIL", "your@email.com", "User")
[System.Environment]::SetEnvironmentVariable("OETC_PASSWORD", "yourpassword", "User")
```

Verify (restart the terminal first for persistent variables to take effect):

```
echo $env:OETC_EMAIL
```

# 2.6 Troubleshooting

If the steps below do not resolve your issue, please open a report on the Support page.

### ⚠ Out of memory during profile building in `build_renewable_profile`

Building renewable profiles with Atlite is memory-intensive. The default configuration uses 4 parallel processes (~20 GB RAM total). On machines with less than 16 GB, reduce `nprocesses` in `config/config.agora.yaml`:

```
atlite:
  nprocesses: 1  # use 1 on machines with < 16 GB RAM
```

### ⚠ Limit the number of cores ( `--cores` )

By default, `-call` ( `--cores all` ) uses every available CPU core, which can exhaust memory and trigger crashes on Linux. Limiting parallelism reduces peak memory and CPU pressure:

If you run into memory pressure, out-of-memory errors, or system instability during the workflow, rerun the same command with fewer cores. A good first step is to reduce to `--cores 2`, and if needed to `--cores 1`.

```
~/PyPSA-IEI$ snakemake --cores 2 all --configfile config/scenarios/config.SE.yaml
```

### ⚠ Windows crashes when `solve_sector_networks` starts

On Windows, you can still start the workflow with multiple cores to speed up preprocessing and postprocessing:

```
~/PyPSA-IEI$ snakemake -call all --configfile config/scenarios/config.SE.yaml
```

If the workflow reaches `solve_sector_networks` and then crashes or becomes unstable, rerun the same target with one core:

```
~/PyPSA-IEI$ snakemake --cores 1 all --configfile config/scenarios/config.SE.yaml
```

Snakemake will reuse completed upstream jobs and continue from the remaining solve and postprocessing steps.

## 2.7 5. Run the Evaluations

To use the analysis scripts developed for this study:

1. Copy the following files into `scripts_analysis/shapes/` :

| File | Source (generated by the pipeline) |
|---|---|
| `country_shapes.geojson` | `resources/<run_name>/country_shapes.geojson` |
| `regions_onshore_elec_s_62.geojson` | `resources/<run_name>/regions_onshore_elec_s_62.geojson` |
| `regions_offshore_elec_s_62.geojson` | `resources/<run_name>/regions_offshore_elec_s_62.geojson` |

   Replace `<run_name>` with the value of `run.name` from your scenario config file (e.g. `2025-MM-DD-branch-2190SEG-SN` ). These files are produced by the `build_shapes` and `cluster_network` Snakemake rules and are available after the pipeline has run at least once.

2. Open `scripts_analysis/analysis_main.py` and update the scenario-specific settings so they match the files created by your run:

```
runs = {
    "SE": "2025-MM-DD-branch-2190SEG-SE",
    "CE": "2025-MM-DD-branch-2190SEG-CE",
    # "scenario_3": "run_name_3",
    # "scenario_4": "run_name_4",
}

sector_opts = {
    "SE": "20SEG-T-H-B-I-A",
    "CE": "20SEG-T-H-B-I-A",
    # "scenario_3": "2190SEG-T-H-B-I-A",
    # "scenario_4": "2190SEG-T-H-B-I-A",
}

postnetwork_prefixes = {
    scen: f"elec_s_62_lv99__{sector_opt}"
    for scen, sector_opt in sector_opts.items()
}

# Reference scenario — used as the baseline in all comparison/residual plots.
# Must be one of the keys defined in `runs`.
sel_scen = "SE"

# No need to edit — overridden automatically by the script during execution.
scenarios_for_comp = ["SE", "CE"]

# Every key in `runs` must have an entry here or the script will crash.
scenario_colors = {
    "SE": "#F58220",
    "CE": "#39C1CD",
    # "scenario_3": "#179C7D",
    # "scenario_4": "#854006",
}
```

   The run name must match the `run.name` value in the corresponding scenario config file and the folder name under `results/` .

   Each `sector_opts` entry must match the solved postnetwork filename suffix exactly, including the temporal resolution and any enabled sector-coupling suffixes. For example:

   - full study run: `2190SEG-T-H-B-I-A`
   - reduced-resolution sector-coupled test: `20SEG-T-H-B-I-A`

   `analysis_main.py` converts these entries into the full postnetwork prefix automatically, so users only need to edit `runs` and `sector_opts` .

Unused scenarios can be commented out.

> **At least two entries required in `runs`**
>
> The cost analysis script generates comparison plots (residual capital costs, residual operational costs) that require a reference scenario and at least one other scenario to compare against. **Running with only one entry in `runs` will cause a crash.**
>
> If you have only one run, duplicate it under a second key as a workaround:
>
> ```
> runs = {
>     "SE":  "2025-MM-DD-branch-2190SEG-SE",
>     "SE2": "2025-MM-DD-branch-2190SEG-SE",  # same run, avoids crash
> }
> sector_opts = {
>     "SE":  "20SEG-T-H-B-I-A",
>     "SE2": "20SEG-T-H-B-I-A",
> }
> sel_scen = "SE"
> scenario_colors = {
>     "SE":  "#39C1CD",
>     "SE2": "#39C1CD",
> }
> ```
>
> The residual/comparison plots will show zero differences, which is expected when comparing a scenario against itself.

3. Execute it:

```
python scripts_analysis/analysis_main.py
```

This will run all evaluations automatically.

---

# 2.8 Documentation Environment (Optional)

To build or preview this documentation locally, create the lightweight docs environment:

```
conda env create -f envs/environment_docs.yaml
conda activate pypsa-iei-docs
mkdocs serve
```

Then open http://127.0.0.1:8000 in your browser.

# 3. Scenario Framework

This page describes the four main scenarios analyzed in the PyPSA-IEI study. Each scenario represents a distinct policy pathway for European energy infrastructure development, defined by two key dimensions:

1. **Infrastructure planning approach**: Cross-sectoral vs. Sectoral silos
2. **Policy perspective**: European view vs. National view

---

## 3.1 Policy Dimensions

### 3.1.1 Dimension 1: Cross-Sectoral vs. Sectoral View

Infrastructure planning can be approached either through **integrated cross-sectoral planning** or through planning in **sectoral silos**.

#### Cross-Sectoral View

Infrastructure planning considers the interaction between sectors (electricity, natural gas, hydrogen) in a consistent, harmonised way. This enables:

- Integrated, cost-optimal grid expansion across all energy carriers
- Efficient sector-coupling approaches
- Minimization of system-wide costs
- Avoidance of grid bottlenecks and overcapacities

**Implementation Details**

The existing grid and already advanced TYNDP projects (i.e., with status "in construction" for electricity, "FID" for gas) form the lower boundary for endogenous transmission capacity expansions. The optimization model can freely expand transmission capacities for electricity, gas, hydrogen, and $CO_2$ networks beyond these baseline projects.

For detailed project lists, see Annex 2 in the published report.

### Sectoral View (Silos)

Infrastructure planning treats energy networks largely independently, with electricity, natural gas, and hydrogen networks planned separately. This reflects current practices like the TYNDP process, but involves risks:

- Unnecessarily high energy system costs
- Potential grid bottlenecks
- Overcapacities in technologies and infrastructure
- Lack of coordination between energy carriers

**Implementation Details**

The existing grid and all sectoral infrastructure projects (TYNDP, H2 Infrastructure Map, and German Hydrogen Core Network) are the only capacity expansions permitted in the system **until 2040**. This reflects network capacity expansion as envisioned by grid planners. After 2040, the optimization model can freely expand transmission capacities.

For electricity networks, this includes less advanced TYNDP projects with maturity status "in permitting" and "under consideration". For complete project listings, see Annex 2 in the published report.

## 3.1.2 Dimension 2: European vs. National View

The second dimension reflects the tension between European internal market integration and national protectionist tendencies.

### European View

The European internal market paradigm supports free trade and optimal resource allocation across borders. The model:

- Can freely optimize the share of imported versus domestically generated electricity and hydrogen
- Maximizes system-wide efficiency
- Leverages regional comparative advantages
- No self-sufficiency constraints are activated

**Implementation Details**

Self-sufficiency constraints are **not activated**. The system freely optimizes imports and exports on both hourly and annual basis to minimize total system costs.

### National View

National perspectives prioritize minimizing energy import dependencies (even from within the EU) by expanding domestic generation and storage capacities. Motivations include:

- Energy security concerns
- Keeping domestic electricity prices low
- Avoiding political dependencies
- National self-determination

This approach can lead to:

- Unnecessarily high system costs
- Excess regional capacities despite sub-optimal generation conditions
- Underutilization of low-cost generation potentials in other countries

**Implementation Details**

Self-sufficiency constraints are **activated** to require high shares of domestic generation for each country. These constraints operate on an **annual basis** (hourly imports/exports remain unrestricted).

For detailed constraint formulation, see Self-Sufficiency Constraints or Annex 1 in the published report.

**Parameterization:**

| Year | Electricity Min. National Generation | Hydrogen Min. National Generation | Export Capacity Buffer |
|---|---|---|---|
| 2030 | 80% | 70% | +10% of demand |
| 2050 | 100% | 70% | +10% of demand |

- Electricity self-sufficiency increases linearly from 80% (2030) to 100% (2050)
- Hydrogen self-sufficiency remains constant at 70%
- Countries must build sufficient capacity to export 10% of their electricity/hydrogen demand

# 3.2 The Four Main Scenarios

| | **European View (no self-sufficiency constraints)** | **National View (with self-sufficiency constraints)** |
|---|---|---|
| **Cross-Sectoral** (free grid expansion) | **CE** *Cross-sectoral, European* Cost-optimal integrated planning | **CN** *Cross-sectoral, National* Integrated planning with self-sufficiency |
| **Sectoral** (fixed grid expansion until 2040) | **SE** *Sectoral, European* Sectoral planning with free trade | **SN** *Sectoral, National* Sectoral planning with self-sufficiency |

# 3.3 Detailed Scenario Descriptions

## 3.3.1 CE Scenario (Cross-sectoral, European view)

**Configuration file:** `config/scenarios/config.CE.yaml`

The **CE scenario** represents fully integrated, cost-optimal European energy infrastructure planning. This scenario serves as the **cost-optimal benchmark** against which other scenarios are compared.

**Vi[redacted] Characteristics**

- ✅ **Free transmission expansion** for all sectors (electricity, gas, hydrogen, $CO_2$)
- ✅ **No self-sufficiency constraints**
- ✅ Existing grid + advanced TYNDP projects form the lower bound
- ✅ Free optimization of import vs. domestic generation shares

**Vi[redacted] Expected Outcomes**

- Cost-optimal generation, storage, and sectorally-integrated grid expansion
- Maximum utilization of regional comparative advantages
- Lowest system-wide costs
- High cross-border energy flows where economically efficient
- Achievement of climate neutrality by 2050 at minimal cost

## 3.3.2 CN Scenario (Cross-sectoral, National view)

**Configuration file:** `config/scenarios/config.CN.yaml`

The **CN scenario** combines integrated infrastructure planning with national self-sufficiency objectives. This scenario explores the **cost of energy sovereignty** within an otherwise optimally integrated system.

**Characteristics**

- ✅ **Free transmission expansion** for all sectors (electricity, gas, hydrogen, $CO_2$)
- ⚠️ **Self-sufficiency constraints activated**
- ✅ Existing grid + advanced TYNDP projects form the lower bound
- ⚠️ High share of domestic generation required (80-100% electricity, 70% hydrogen)

**Expected Outcomes**

- Sectorally-integrated grid expansion within national constraints
- High level of national energy supply
- Low import dependence (including intra-European)
- Higher domestic generation and storage capacities
- Increased system costs compared to CE due to sub-optimal resource allocation
- Underutilization of cross-border transmission despite available capacity

---

## 3.3.3 SE Scenario (Sectoral, European view)

**Configuration file:** `config/scenarios/config.SE.yaml`

The **SE scenario** reflects current infrastructure planning practices (e.g., TYNDP) with European market integration. This scenario evaluates the **cost of sectoral planning** under current institutional frameworks.

**Characteristics**

- ⚠️ **Fixed transmission expansion until 2040** based on:
  - TYNDP electricity projects (including "in permitting" and "under consideration")
  - TYNDP gas projects
  - H2 Infrastructure Map
  - German Hydrogen Core Network
- ✅ **No self-sufficiency constraints**
- ✅ Free optimization of import vs. domestic generation shares
- ✅ Free transmission expansion after 2040

**Expected Outcomes**

- Good EU-wide coordination on trade and generation
- Sub-optimal infrastructure due to sectoral planning silos
- Networks may be insufficient or misaligned for integrated sector-coupling
- Potential grid bottlenecks or stranded assets
- Higher costs than CE due to lack of integrated infrastructure optimization
- Increased reliance on generation and storage to compensate for grid constraints

---

## 3.3.4 SN Scenario (Sectoral, National view)

**Configuration file:** `config/scenarios/config.SN.yaml`

The **SN scenario** represents the most constrained policy pathway, combining sectoral planning with national protectionism. This scenario represents the **worst-case policy pathway** where both infrastructure planning and trade policies work against system optimization.

**Characteristics**

- ⚠️ **Fixed transmission expansion until 2040** (same as SE)
- ⚠️ **Self-sufficiency constraints activated**
- ⚠️ High share of domestic generation required (80-100% electricity, 70% hydrogen)
- ⚠️ Limited cross-border coordination
- ✅ Free transmission expansion after 2040

**Expected Outcomes**

- Uncoordinated expansion of generation and grid infrastructure
- Highest system costs among all scenarios
- Significant overcapacities in some regions, bottlenecks in others
- Sub-optimal use of renewable resources
- Stranded assets due to fixed, uncoordinated network planning
- Low cross-border energy flows despite physical interconnection

---

## 3.4 Sensitivity Scenarios

In addition to the four main scenarios, several **sensitivity analyses** were conducted based on the CE scenario to test robustness:

| Scenario | Configuration File | Description |
|---|---|---|
| **CE (base)** | `config.CE.yaml` | Reference scenario |
| **CE - No Gas/Oil** | `config.CE_no_gas_no_oil.yaml` | Fossil gas and oil phase-out |
| **CE - Flexibility** | `config.CE_flexibility.yaml` | Limited flexibility options |
| **CE - Expensive Compensation** | `config.CE_expensive_compensation.yaml` | Higher balancing costs |
| **CE - TransHyDE S2** | `config.CE_TranshydeS2.yaml` | Alternative industrial demand profile |

## 3.5 Running Scenarios

To execute a specific scenario, use Snakemake with the corresponding configuration file:

```
snakemake -call all --configfile config/scenarios/config.CE.yaml
```

Replace `config.CE.yaml` with the desired scenario configuration file.

For detailed execution instructions, see the Installation page.

## 3.6 Key Differences Summary

| Feature | CE | CN | SE | SN |
|---|---|---|---|---|
| Grid expansion (until 2040) | Free | Free | Fixed (TYNDP) | Fixed (TYNDP) |
| Grid expansion (after 2040) | Free | Free | Free | Free |
| Self-sufficiency constraints | ❌ No | ✅ Yes | ❌ No | ✅ Yes |
| Cross-border optimization | Full | Limited | Full | Limited |
| Expected system cost | Lowest | Medium-low | Medium-high | Highest |
| Infrastructure planning | Integrated | Integrated | Sectoral silos | Sectoral silos |
| Trade policy | European market | National focus | European market | National focus |

## 3.7 Related Documentation

- Self-Sufficiency Constraints — Technical implementation details
- TYNDP Electricity Projects — Fixed electricity network projects
- Gas Networks — TYNDP gas pipeline implementation
- Hydrogen Networks — H2 Infrastructure Map and Core Network
- Expansion Limits — Constraints on capacity expansion

## 3.8 References

For comprehensive methodology and results analysis, please refer to:

> "*Integrated Infrastructure Planning and 2050 Climate Neutrality: Deriving Future-Proof European Energy Infrastructures*" Fraunhofer IEG, Fraunhofer ISI, and d-fine (November 2025) Download report (PDF)

# 4. ⚙️ Model Functionality

## 4.1 Model Functionality Overview

This section describes the key extensions and modifications made to the PyPSA-Eur v0.10.0 base model for the PyPSA-IEI study. Each sub-page covers one topic with a description, the relevant Snakemake rule(s), and code/config snippets for reproducibility and future modification.

### 4.1.1 Contents

| Topic | Description |
|---|---|
| Costs | Custom cost assumptions from config and technology-data |
| Lifetimes | Country- and plant-level powerplant lifetime adjustments |
| Gas | TYNDP gas pipelines, Russian import removal |
| Hydrogen | Wasserstoffkernnetz, ENTSO-G network, retrofitting |
| $CO_2$ Pipeline Network | Spatial $CO_2$ tracking, bidirectional pipeline links, sequestration caps |
| Industrial Demand | TransHyDe-based industrial energy demand |
| Imports | $H_2$, syngas, and synfuel import nodes |
| Losses | Transmission efficiency and compression losses |
| TYNDP Electricity Projects | Cross-border TYNDP transmission enforcement |
| National Grid Plans | National transmission expansion factors per country |
| Myopic Optimization | Brownfield capacity accounting across horizons |
| Expansion Limits | Min/max capacity constraints per carrier and country |
| Self-Sufficiency Constraints | Minimum/maximum self-sufficiency targets per region |
| Overall Minimum Capacities | System-wide minimum capacity per carrier |
| Capacity Reserve Margin | Reserve margin above peak demand for conventional generation |
| Import Infrastructure Retrofitting | Gas import infrastructure reuse for $H_2$ and syngas |
| Curtailment Mode | Explicit renewable curtailment modeling |

# 4.2 📐 Parameters & Configuration

## 4.2.1 Costs

### Overview

The model uses cost assumptions from technology-data v0.9.0 as a baseline (stored in `data/costs_{planning_horizon}.csv`). Selected investment costs are **overridden per planning horizon** using values defined in `config/config.agora.yaml`.

---

### Snakemake Rule

**Rule:** `update_costs_csv`
**Script:** `scripts/update_costs_csv.py`

This rule reads the baseline costs CSV and updates specific technology investment costs before the network is built.

---

### How It Works

**scripts/update_costs_csv.py**

```
def insert_new_costs(costs_file, invest_update, filename):
    costs = pd.read_csv(costs_file).set_index('technology')
    for key, investment in invest_update.items():
        mask = (costs.parameter == "investment") & (costs.index == key)
        costs.loc[mask, 'value'] = investment
    costs.to_csv(filename)

# Called with per-horizon values from config:
investment_update = snakemake.params.invest_update[int(snakemake.wildcards.planning_horizons)]
insert_new_costs(snakemake.input.costs, investment_update, snakemake.output.costs)
```

---

### Technologies Updated in All Scenarios

These investment costs are overridden for **every scenario** and every planning year:

| Technology | Config key |
|---|---|
| HVDC overhead line | `HVDC overhead` |
| HVDC submarine cable | `HVDC submarine` |
| HVAC overhead line | `HVAC overhead` |
| $H_2$ electrolysis | `electrolysis` |
| $CO_2$ pipeline | `CO2 pipeline` |
| $CO_2$ submarine pipeline | `CO2 submarine pipeline` |

---

### Additional Technologies (CE Flexibility Scenario Only)

In the CE flexibility scenario ( `config.CE_flexibility.yaml` ), investment costs for the following heat-sector technologies are set to **20% above the technology-data v0.9.0 baseline** for every planning year:

| Technology (config key) |
|---|
| `central air-sourced heat pump` |
| `central ground-sourced heat pump` |
| `decentral air-sourced heat pump` |
| `decentral ground-sourced heat pump` |
| `central solid biomass CHP` |
| `central gas CHP` |
| `micro CHP` |
| `biomass CHP capture` |
| `central water tank storage` |
| `decentral water tank storage` |
| `central resistive heater` |
| `decentral resistive heater` |

Example for 2030:

| Technology | Unit | Baseline (tech-data) | CE Flexibility (+20%) |
|---|---|---|---|
| `central air-sourced heat pump` | EUR/kW_th | 906.1 | 1087.3 |
| `decentral air-sourced heat pump` | EUR/kW_th | 899.5 | 1079.4 |
| `central gas CHP` | EUR/kW | 592.6 | 711.1 |
| `micro CHP` | EUR/kW_th | 7841.7 | 9410.1 |
| `central water tank storage` | EUR/kWh | 0.576 | 0.691 |

---

## How to Modify the Cost Parameters

#### STEP 1: FIND THE CORRECT TECHNOLOGY NAME

Technology names must exactly match the `technology` column in `data/costs_<year>.csv`, which in turn comes from technology-data v0.9.0.

#### STEP 2: EDIT THE SCENARIO CONFIG FILE

Add or update the technology under `costs.investment` in `config/config.agora.yaml` (base config, applies to all scenarios) or in a scenario-specific file to override only for that scenario. Units depend on the technology — check the `unit` column in the costs CSV first.

**config/config.agora.yaml or config/scenarios/config.*.yaml**

```yaml
costs:
  investment:
    2020:
      HVDC overhead: 2000.0        # EUR/MW/km – IEA 2023
      HVDC submarine: 2000.0       # EUR/MW/km
      HVAC overhead: 500.0         # EUR/MW/km
      electrolysis: 1260.0         # EUR/kW_e – IEA Global Hydrogen Review 2023
      CO2 pipeline: 13000.0        # EUR/(tCO2/h)/km – Danish Energy Agency
      CO2 submarine pipeline: 33000.0  # EUR/(tCO2/h)/km – Danish Energy Agency
    2025:
      HVDC overhead: 2000.0
      ...
```

## 4.2.2 Powerplant Lifetimes

### Overview

Standard powerplantmatching data does not always reflect country-specific or plant-specific decommissioning decisions. This model updates powerplant lifetimes based on a custom CSV, incorporating data from:

- Beyond Fossil Fuels
- Reuters
- Forum Energii (2024)

---

### Snakemake Rule

**Rule:** `build_powerplants`
**Script:** `scripts/build_powerplants.py`
**Data file:** `data/powerplant_lifetime.csv`

---

### How It Works

The `build_powerplants` rule retrieves the standard powerplant database from powerplantmatching and then calls `manipulate_lifetimes()` to apply corrections from the custom CSV:

**scripts/build_powerplants.py**

```
ppl = manipulate_lifetimes(ppl, snakemake.input.new_ppl_lifetimes)
```

The function handles four cases based on the `Name` and `status` columns:

**scripts/build_powerplants.py**

```
def manipulate_lifetimes(ppl_lifetime, path_lifetimes):
    lifetime_data = pd.read_csv(path_lifetimes)

    # Four adjustment categories:
    ppl_specific_lifetimes_limit  = lifetime_data.query("(Name!='all') & (status=='limit')")
    country_level_lifetimes_limit = lifetime_data.query("(Name=='all') & (status=='limit')")
    ppl_specific_lifetimes_ext    = lifetime_data.query("(Name!='all') & (status=='extend')")
    country_level_lifetimes_ext   = lifetime_data.query("(Name=='all') & (status=='extend')")

    # For 'limit': replace DateOut only if the new date is *earlier* than the existing one
    # For 'extend': replace DateOut only if the new date is *later* than the existing one
    # Applied first at country level, then at individual plant level
```

Adjustments are applied at two levels:

- **Country-level** ( `Name == 'all'` ): All plants of a given fuel type in a country get their `DateOut` updated (e.g. coal phase-out in Germany by 2030)
- **Plant-level** ( `Name != 'all'` ): Individual plants receive specific retirement years based on confirmed announcements

The logic is conservative: a `limit` entry only takes effect if the plant would have lived *longer* than specified; an `extend` entry only takes effect if the plant would have retired *earlier* than specified.

---

### Data Format

`data/powerplant_lifetime.csv` columns (in order):

| Column | Description |
|---|---|
| `Country` | ISO2 country code |
| `Fueltype` | e.g. `Lignite` , `Hard Coal` , `Nuclear` |
| `DateOut` | Target retirement year |
| `Name` | Plant name for plant-level rules; `all` for country-level rules |
| `status` | `limit` (force earlier retirement) or `extend` (force later retirement) |
| `Reference` | Data source citation |

---

### Current Data in `powerplant_lifetime.csv`

#### COAL / LIGNITE PHASE-OUTS (COUNTRY-LEVEL `limit`)

All entries use `Name = all` and `status = limit`, sourced from Beyond Fossil Fuels except Poland which uses Forum Energii (25.11.2024):

| Country | Fueltype | DateOut |
|---|---|---|
| PL | Lignite + Hard Coal | 2044 |
| DE | Lignite + Hard Coal | 2038 |
| BG | Lignite + Hard Coal | 2038 |
| HR, CZ, SI | Lignite + Hard Coal | 2033 |
| ME | Lignite + Hard Coal | 2035 |
| RO | Lignite + Hard Coal | 2032 |
| MK | Lignite + Hard Coal | 2030 |
| NL, FI | Lignite + Hard Coal | 2029 |
| DK | Lignite + Hard Coal | 2028 |
| FR, HU, IT | Lignite + Hard Coal | 2027 |
| GR | Lignite + Hard Coal | 2026 |
| ES, IE | Lignite + Hard Coal | 2025 |
| SK | Lignite + Hard Coal | 2024 |
| GB | Lignite + Hard Coal | 2024 |
| PT | Lignite + Hard Coal | 2021 |
| AT, SE | Lignite + Hard Coal | 2020 |
| BE | Lignite + Hard Coal | 2016 |

#### NUCLEAR LIFETIME EXTENSIONS (PLANT-LEVEL `extend`)

French plants extended per Macron's 2022 announcement (Reuters), Belgian reactors extended per the 2023 Engie agreement (Reuters):

| Country | Plant | DateOut |
|---|---|---|
| FR | Civaux | 2057 |
| FR | Nogent, Belleville | 2047 |
| FR | Golfech, Penly | 2050 |
| FR | St Alban, Flamanville | 2045 |
| FR | Cruas | 2043 |
| FR | Paluel | 2044 |
| FR | Cattenom | 2038 |
| FR | Blayais, Gravelines, Dampierre | 2040–2041 |
| FR | Chooz | 2027 |
| FR | Bugey | 2032 |
| FR | Chinon | 2023 |
| BE | Doel, Tihange | 2035 |

---

### How to Modify

The active lifetime file is set in the config under `electricity`:

**config/config.agora.yaml**

```yaml
electricity:
  new_powerplant_lifetimes: data/powerplant_lifetime.csv
```

You have two options depending on whether the change should apply to **all runs** or only to **specific scenarios**.

---

#### OPTION 1: EDIT THE SHARED FILE (APPLIES TO ALL RUNS)

Edit `data/powerplant_lifetime.csv` directly. All runs that reference this file will pick up the change.

**Limit a country-wide fuel type (phase-out)**

Set `Name` to `all` and `status` to `limit`. All plants of that fuel type in that country will retire no later than `DateOut`:

```
Country,Fueltype,DateOut,Name,status,Reference
DE,Lignite,2030,all,limit,custom
PL,Hard Coal,2040,all,limit,custom
```

### Extend a country-wide fuel type (lifetime extension)

```
Country,Fueltype,DateOut,Name,status,Reference
CZ,Nuclear,2050,all,extend,custom
```

### Override a specific plant

Use the exact plant name as it appears in the powerplantmatching database:

```
Country,Fueltype,DateOut,Name,status,Reference
PL,Lignite,2036,Bełchatów,limit,custom
CZ,Nuclear,2050,Dukovany,extend,custom
```

---

#### OPTION 2: CREATE A SCENARIO-SPECIFIC FILE (APPLIES TO SELECTED RUNS ONLY)

1. Copy the base file and give it a descriptive name:

```
cp data/powerplant_lifetime.csv data/powerplant_lifetime_nuclear_extension.csv
```

2. Edit the copy with the scenario-specific adjustments.
3. Point to it in the relevant scenario config under `config/scenarios/`:

**config/scenarios/config.MY_SCENARIO.yaml**

```yaml
electricity:
  new_powerplant_lifetimes: data/powerplant_lifetime_nuclear_extension.csv
```

This overrides only `new_powerplant_lifetimes` for that scenario; all other config values remain inherited from `config.agora.yaml`.

> **Tip**
>
> This approach is recommended when you want to compare scenarios with different phase-out assumptions — e.g. a baseline with the default file and a sensitivity with a more aggressive coal exit date — without touching the shared file.

## 4.2.3 Transmission Losses

### Overview

Transmission losses are modelled by splitting each bidirectional link into two unidirectional links with `efficiency < 1.0`. The function `lossy_bidirectional_links()` is applied to every carrier listed under `sector.transmission_efficiency` in the config. DC links carry a static converter loss and a length-dependent resistive loss. $H_2$ and gas pipelines carry compression losses, drawn as electricity from the sending node. TYNDP gas pipelines receive the same loss parameters as standard gas pipelines.

---

### Snakemake Rule

**Rule:** `prepare_sector_network`
**Script:** `scripts/prepare_sector_network.py` — function `lossy_bidirectional_links()`

---

### Carriers with Losses

| Carrier | `efficiency_static` | `efficiency_per_1000km` | `compression_per_1000km` |
|---|---|---|---|
| DC | 0.98 | 0.977 | — |
| H2 pipeline | — | 1.0 | 0.019 |
| gas pipeline | — | 1.0 | 0.010 |
| electricity distribution grid | 1.0 | — | — |

---

### Implementation

Each bidirectional link is split into a forward link (efficiency applied by length and static factor) and a reversed copy (`capital_cost = 0`, `length = 0`). For $H_2$ and gas pipelines, a secondary electricity bus (`bus2`) is attached at the sending node and `efficiency2` is set to a negative value proportional to pipeline length, representing compression energy consumption.

The function is called in a loop over all configured carriers:

**scripts/prepare_sector_network.py**

```
for k, v in options["transmission_efficiency"].items():
    lossy_bidirectional_links(n, k, v)
```

TYNDP gas pipelines receive an additional dedicated call using the same `gas pipeline` efficiencies:

**scripts/prepare_sector_network.py**

```
losses = snakemake.config["sector"]["transmission_efficiency"]["gas pipeline"]
lossy_bidirectional_links(n, "gas pipeline tyndp", losses)
```

### Configuration

Efficiency parameters are set under `sector.transmission_efficiency`:

**config/config.agora.yaml**

```yaml
sector:
  transmission_efficiency:
    DC:
      efficiency_static: 0.98
      efficiency_per_1000km: 0.977
    H2 pipeline:
      efficiency_per_1000km: 1      # 0.979 commented out
      compression_per_1000km: 0.019
    gas pipeline:
      efficiency_per_1000km: 1      # 0.977 commented out
      compression_per_1000km: 0.01
    electricity distribution grid:
      efficiency_static: 1          # 0.97 commented out
```

**Note**

`solving.options.transmission_losses` does **not** control whether losses are applied. It only triggers removal of AC lines with `num_parallel == 0` that would conflict with the loss-aware solver:

```yaml
solving:
  options:
    transmission_losses: 2
```

### How to Change Transmission Loss Values

Edit the efficiency values under `sector.transmission_efficiency` in `config/config.agora.yaml` or a scenario override file. Setting `efficiency_per_1000km: 1` and `compression_per_1000km: 0` for a carrier effectively disables losses for that carrier.

## 4.2.4 Flexible Time Segmentation

### Overview

Full-year hourly time series (8760 snapshots) make sector-coupled optimisation extremely slow. To keep solve times tractable, the model aggregates snapshots into a smaller number of **representative segments** using the tsam library. The number of segments is controlled via the `sector_opts` wildcard (e.g. `2190SEG`, `20SEG`) and can be changed freely for each run without touching any script.

Two simpler alternatives also exist:

| Method | `sector_opts` example | Effect |
|---|---|---|
| Temporal segmentation (tsam) | `2190SEG` | Aggregate 8760 h → N representative segments |
| Hourly resampling | `4h` | Average every 4 hours → 2190 snapshots |
| No aggregation | *(no suffix)* | Keep all hourly snapshots |

---

### Snakemake Rules

**Rule:** `time_aggregation` (in `rules/build_sector.smk`)
**Script:** `scripts/time_aggregation.py`

This rule runs **before** `prepare_sector_network` and writes a CSV of snapshot weightings. The sector-coupled network then reads that CSV and applies the aggregation.

**Rule:** `prepare_sector_network`
**Script:** `scripts/prepare_sector_network.py` — function `set_temporal_aggregation()`

---

### How It Works

#### STEP 1 — COMPUTE SNAPSHOT WEIGHTINGS (`time_aggregation.py`)

All time-varying data is collected (generator profiles, load, heat demand, solar thermal) and normalised to the unit interval. `tsam` then partitions the 8760-hour year into N contiguous segments, each described by a representative hour and a duration weight:

**scripts/time_aggregation.py**

```
agg = tsam.TimeSeriesAggregation(
    df,
    hoursPerPeriod=len(df),   # treat the whole year as one period
    noTypicalPeriods=1,
    noSegments=segments,       # e.g. 2190 or 20
    segmentation=True,
    solver=snakemake.params.solver_name,
)
```

```
agg = agg.createTypicalPeriods()

weightings = agg.index.get_level_values("Segment Duration")
offsets = np.insert(np.cumsum(weightings[:-1]), 0, 0)
snapshot_weightings = n.snapshot_weightings.loc[n.snapshots[offsets]].mul(
    weightings, axis=0
)
snapshot_weightings.to_csv(snakemake.output.snapshot_weightings)
```

The result is a CSV with one row per segment, indexed by the representative timestamp and holding the duration of that segment in hours.

#### STEP 2 — APPLY TO THE SECTOR-COUPLED NETWORK (`prepare_sector_network.py`)

`set_temporal_aggregation()` reads the CSV and reindexes all time-dependent network data (profiles, loads, snapshot weightings) to the reduced set of snapshots:

**scripts/prepare_sector_network.py**

```
snapshot_weightings = pd.read_csv(
    snakemake.input.snapshot_weightings, index_col=0, parse_dates=True
)
set_temporal_aggregation(n, snapshot_weightings)
```

---

## Configuration

#### VIA THE `sector_opts` WILDCARD

The number of segments is set in `sector_opts` in the scenario config file. The `SEG` suffix is parsed at runtime — no code change is needed:

**config/scenarios/config.SE.yaml**

```
scenario:
  sector_opts:
  - 2190SEG-T-H-B-I-A   # full study resolution
```

| `sector_opts` value | Segments | Intended use |
|---|---|---|
| `2190SEG-T-H-B-I-A` | 2190 | Full study (publication results) |
| `20SEG-T-H-B-I-A` | 20 | Pipeline test with sector coupling |
| `20SEG` | 20 | Electricity-only pipeline test |

#### VIA THE STATIC CONFIG (ALTERNATIVE)

`snapshots.segmentation` in `config/config.agora.yaml` can also trigger segmentation. It is `false` by default — all scenarios use the wildcard approach instead:

**config/config.agora.yaml**

```
snapshots:
  start: "2013-01-01"
  end: "2014-01-01"
  inclusive: 'left'
```

```
  resolution: false      # hourly resampling, e.g. "4h" — disabled by default
  segmentation: false    # static segment count — disabled by default
```

The wildcard always takes precedence over the static config value.

---

### How to Change the Number of Segments

Edit `sector_opts` in the relevant scenario config:

**config/scenarios/config.SE.yaml**

```
scenario:
  sector_opts:
  - 20SEG-T-H-B-I-A   # fewer segments → faster, less accurate
```

> **Warning**
>
> Results produced with different segment counts are **not directly comparable**. The study uses `2190SEG` for all published results. Use reduced counts (e.g. `20SEG`) only for pipeline testing.

# 4.3 🔌 Network Infrastructure

## 4.3.1 Gas Network

### Overview

The gas network is a **standard PyPSA-Eur feature**. The existing European transmission infrastructure is reconstructed from two open datasets and added to the model as PyPSA `Link` components. This page describes how that base infrastructure is represented, followed by the two modifications made in this project relative to base PyPSA-Eur:

1. **Russian gas imports are removed** in all scenarios by default
2. **TYNDP gas pipeline projects** are processed and added to the model

---

### Existing Infrastructure (PyPSA-Eur)

#### DATA SOURCES

| Dataset | What it provides |
|---|---|
| **SciGRID_gas** (scigrid.de) | European pipe geometry, diameter, pressure, bidirectionality, underground storage locations |
| **GEM** (Global Energy Monitor) | LNG terminal locations and capacities, onshore production sites |

#### BUILD PIPELINE

Three Snakemake rules prepare the existing network before it reaches `prepare_sector_network`:

1. **`build_gas_network`** (`scripts/build_gas_network.py`) — loads the SciGRID_gas GeoJSON (`IGGIELGN_PipeSegments.geojson`); infers pipe capacity from diameter using the piecewise-linear formula from the European Hydrogen Backbone report (e.g. 500 mm → ~1.5 GW, 900 mm → ~11.25 GW); corrects outlier capacities and lengths; flags short pipes (<10 km) as bidirectional.
2. **`cluster_gas_network`** (`scripts/cluster_gas_network.py`) — spatially joins both pipe endpoints to model bus regions; recalculates length as haversine × 1.25 (routing factor); drops intra-region pipes; aggregates parallel pipes on the same corridor by summing `p_nom`.
3. **`build_gas_input_locations`** (`scripts/build_gas_input_locations.py`) — assembles per-node supply capacity tables for LNG terminals, cross-border pipeline entry points, onshore production, and underground storage.

When `H2_retrofit: true`, gas pipes become candidates for conversion to hydrogen transport — their capital cost is reduced to a small decommissioning penalty and capacity is freed for `"H2 pipeline retrofitted"` links on the same corridors.

### Russian Import Removal

**Rule:** `build_gas_input_locations` **Script:** `scripts/build_gas_input_locations.py`

The base config sets `import_from_russia: false` under `sector:`, so Russian pipeline entry points are excluded in all scenarios. If re-enabled but `nordstream` is disabled, only the Nord Stream pipeline is excluded:

**scripts/build_gas_input_locations.py**

```
if not snakemake.params.import_from_russia:
    entry = entry.loc[~(entry.from_country == "RU")]
elif not snakemake.params.nordstream:
    entry = entry.loc[entry.id != "INET_BP_63"]
```

### TYNDP Gas Pipelines

**Rule:** `build_tyndp_gas_pipes`
**Script:** `scripts/build_tyndp_gas_pipes.py`
**Input file:** `data/gas_network/TYNDP_Gas_Interconnectors.xlsx`

#### INPUT DATA FORMAT

The Excel file contains the following columns (rows with any missing value in these columns are dropped):

| Column | Type | Description |
|---|---|---|
| `Code` | string | Unique project identifier (e.g. `TRA-N-7`) |
| `Project Name` | string | Human-readable project name; combined with `Code` into a `tag` field |
| `Maturity Status` | string | One of `FID`, `Advanced`, `Less-Advanced` — used for scenario-dependent filtering |
| `Diameter (mm)` | float | Pipe inner diameter; converted to capacity via the same piecewise-linear formula used for existing SciGRID_gas pipes |
| `Length (km)` | float | Pipeline length in km |
| `PCI 5th List` | string | `Yes` or `No` — whether the project is on the EU Projects of Common Interest (PCI) 5th list |
| `Project Commissioning Year Last` | int | Latest commissioning year; determines which planning horizon the pipe is built in |
| `Start` | string | Start location — either a 2-letter country code (e.g. `DE`) for countries with a single cluster, or DMS coordinates (e.g. `51°30'N 000°07'E`) for countries with multiple clusters |
| `End` | string | End location — same format as `Start` |

Additional columns present in the file (`Project Description`, `Capacities`, `Reference`) are read but dropped before output.

**Adding a new project**

Copy an existing row, assign a new `Code`, fill in all required columns, and set `Maturity Status` to the appropriate level. The script will automatically:

- convert `Diameter (mm)` to MW capacity
- resolve 2-letter country codes or DMS coordinates to model cluster names
- drop the project if both endpoints fall within the same cluster

TYNDP gas pipeline project data is processed and mapped to model cluster regions using geographic coordinates:

**scripts/build_tyndp_gas_pipes.py**

```
# Load TYNDP project list
tyndp_raw_data = pd.read_excel(snakemake.input.tyndp_gas_projects)

# Remove projects with missing critical data
tyndp_raw_data.dropna(subset=[
```

```
    'Code', 'Project Name', 'Maturity Status', 'Diameter (mm)',
    'Length (km)', 'PCI 5th List', 'Project Commissioning Year Last',
    'Start', 'End'
], inplace=True)

# Convert DMS coordinates to cluster names
def coordinates_to_cluster(coordinates):
    ...
    point = Point(dd_lon, dd_lat)
    return regions.loc[regions['geometry'].contains(point), 'name'].values[0]
```

Pipe capacity is derived from diameter using the `diameter_to_capacity` function (from `build_gas_network.py`). Projects entirely within one cluster are dropped. The output is saved to CSV for use in `prepare_sector_network`.

#### SCENARIO-DEPENDENT PROJECT SELECTION

TYNDP projects are filtered by maturity status via the scenario config:

| Scenario | `allowed_statuses` |
|---|---|
| CE, CN | `FID` only |
| SE, SN | `FID`, `Advanced`, `Less-Advanced` (default — no override in scenario config) |

**config/scenarios/config.CE.yaml**

```
policy_plans:
  include_tyndp_gas:
    enable: true
    allowed_statuses:
      - FID  # Final Investment Decision
```

---

### Pipeline Extendability

```
graph TD
    A[(SciGRID_gas)] -->|"build_gas_network\ncluster_gas_network"| B["gas pipeline\n– existing –"]
    B -->|"H2_retrofit: true\np_nom_max = p_nom"| C["H2 pipeline retrofitted\np_nom_max ≤ 0.6 × gas p_nom"]

    D["k-edge augmentation\ngas_network_connectivity_upgrade"] --> E["gas pipeline new\n– connectivity gaps –"]

    F[(TYNDP Excel)] -->|"build_tyndp_gas_pipes"| G["gas pipeline tyndp\np_nom_extendable=False"]

    style B fill:#BDD7EE,color:#1a1a1a
    style C fill:#D9B3F0,color:#1a1a1a
    style E fill:#C6E0B4,color:#1a1a1a
    style G fill:#FCE4D6,color:#1a1a1a
```

**PyPSA capacity parameters**

- `p_nom` — the nominal (installed) capacity of a link in MW. For existing pipes this equals the current physical capacity read from the dataset.
- `p_nom_extendable` — if `True`, the optimizer may change the capacity; if `False`, the capacity is fixed at `p_nom` and no investment decision is made.
- `p_nom_min` / `p_nom_max` — bounds on the capacity the optimizer can choose. `p_nom_max = p_nom` means the pipe can never grow beyond its current size; `p_nom_min = 0` means it can shrink all the way to zero (i.e. be decommissioned).

| Type | Carrier | Source | `p_nom_extendable` |
|---|---|---|---|
| Existing pipes | `gas pipeline` | SciGRID_gas (clustered) | Depends on `H2_retrofit` and `wasserstoff_kernnetz.optimize_after` — see below |
| New connectivity pipes | `gas pipeline new` | k-edge augmentation | Always `True` — controlled by `include_tyndp_gas.optimize_after` |
| TYNDP projects | `gas pipeline tyndp` | TYNDP Excel file | Always `False` — fixed at commissioned capacity |

#### EXISTING PIPES (`gas pipeline`)

Behaviour depends on whether `H2_retrofit` is enabled:

- **`H2_retrofit: true`** (default): Existing pipes get a small decommissioning capital cost (`0.1 €/MW/km/a`), `p_nom_max = p_nom` and `p_nom_min = 0`. The pipe capacity is therefore **capped at its current value** — it can never grow. Instead the optimizer can *reduce* it toward zero, freeing that capacity for `H2 pipeline retrofitted` links on the same corridors. Two separate mechanisms reduce gas pipe capacity:
  a. **Exogenous reduction** (`scripts/modify_prenetwork.py`): when `wasserstoff_kernnetz` is enabled, gas pipe `p_nom` is directly reduced by the `removed_gas_cap` of all Wasserstoffkernnetz (hydrogen core network) pipes commissioned up to the current horizon — this happens unconditionally, before any optimization.
  b. **Optimizer reduction** (`scripts/prepare_sector_network.py`): whether the optimizer can *additionally* reduce gas capacity is controlled by `wasserstoff_kernnetz.optimize_after` via `p_nom_extendable` — fixed until that year, freely reducible after. The same `optimize_after` parameter also gates all **H2 pipeline** extendability (carriers containing `"H2 pipeline"`) in `scripts/modify_prenetwork.py`.
- **`H2_retrofit: false`**: Pipes are fully extendable with no upper bound (`p_nom_max = inf`, `p_nom_min = p_nom`, full capital cost charged) — capacity can grow freely.

#### NEW CONNECTIVITY PIPES (`gas pipeline new`)

Added via k-edge augmentation to ensure `gas_network_connectivity_upgrade`-connectivity of the gas bus graph. Always extendable. Whether they are added at all is controlled by `include_tyndp_gas.optimize_after`: set to `false` to suppress them entirely, `true` (default) to always add

them. An integer value has no practical gating effect due to how the logic is implemented — only `false` prevents new pipes from being added.

#### TYNDP GAS PROJECTS (`gas pipeline tyndp`)

Commissioned at fixed capacity (`p_nom_extendable=False`) in the planning horizon their commissioning year falls into. The `optimize_after` key in the config has **no effect** on these — it controls new connectivity pipes only.

> **No `optimize_after` for gas**
>
> Unlike TYNDP electricity projects and national grid plans, TYNDP gas pipelines are always added as **fixed capacity** (`p_nom_extendable=False`). The `optimize_after` key exists in the config but has no effect — commissioned projects are simply built in the horizon they fall into and cannot be further optimized.

---

### How to Modify the Gas Network

#### CHANGE WHICH TYNDP PROJECTS ARE INCLUDED

Set `allowed_statuses` in the scenario config file. To allow all maturity levels, omit the key entirely:

**config/scenarios/config.CE.yaml**

```yaml
policy_plans:
  include_tyndp_gas:
    enable: true
    allowed_statuses:
      - FID
      - Advanced
      - Less Advanced
```

#### CONTROL RUSSIAN IMPORT HANDLING

Override the parameter in the scenario config:

```yaml
sector:
  import_from_russia: false  # set to true to allow Russian pipeline gas
  nordstream: false          # set to true to include Nord Stream specifically
```

To add custom gas pipeline projects, extend the TYNDP Excel input file (`data/gas_network/TYNDP_Gas_Interconnectors.xlsx`) — see the Input data format table above for the required columns.

## 4.3.2 Hydrogen Network

### Overview

$H_2$ pipelines are present in **all scenarios**, but the treatment differs:

| Feature | CE / CN | SE / SN |
|---|---|---|
| Retrofit of existing gas pipes to $H_2$ | ✅ endogenous | ✅ endogenous |
| New-build $H_2$ pipelines | ✅ endogenous from 2020 | ✅ endogenous from 2020 |
| *Wasserstoffkernnetz* + European $H_2$ infrastructure map | ✗ | ✅ pre-committed until 2040 |

In **CE and CN**, the $H_2$ network is built entirely **endogenously** — the optimiser decides in every planning horizon which gas pipes to retrofit and which new $H_2$ pipelines to build, with no pre-committed infrastructure. Endogenous $H_2$ pipelines and gas-pipe retrofitting are **standard PyPSA-Eur v0.10.0 features**.

In **SE and SN**, the German *Wasserstoffkernnetz* (FNB-Gas) and the European hydrogen infrastructure map are layered on top as pre-committed infrastructure: the kernnetz pipelines themselves are fixed (non-extendable) until 2040, after which they become optimisable. Endogenous $H_2$ pipeline expansion and gas retrofitting remain active in all horizons, exactly as in CE and CN. This is a **PyPSA-IEI addition**, adapted from PyPSA-Ariadne.

---

### $H_2$ Pipeline Topology (all scenarios)

New $H_2$ pipeline candidates are generated by `create_network_topology()` using the combined corridor set of existing **AC/DC electricity lines** and **CH4 gas pipelines**. This is not a free all-to-all graph — a candidate link is only created where transmission infrastructure already exists. The function is called in `add_storage_and_grids()` in `prepare_sector_network.py`:

**scripts/prepare_sector_network.py**

```
h2_pipes = create_network_topology(
    n, "H2 pipeline ", carriers=["DC", "gas pipeline"]
)
n.madd("Link", h2_pipes.index, ..., p_nom_extendable=True, carrier="H2 pipeline")
```

Gas-pipe retrofit candidates follow the same topology, with `p_nom_max = p_nom_CH4 × H2_retrofit_capacity_per_CH4` (default 0.6).

---

### Wasserstoffkernnetz (SE / SN only)

#### SNAKEMAKE RULES

| Rule | Script | Purpose |
|---|---|---|
| `build_wasserstoff_kernnetz` | `scripts/build_wasserstoff_kernnetz.py` | Process and merge FNB-Gas + ENTSO-G data |
| `cluster_wasserstoff_kernnetz` | `scripts/cluster_wasserstoff_kernnetz.py` | Map $H_2$ network to model clusters |
| `modify_prenetwork` | `scripts/modify_prenetwork.py` | Integrate $H_2$ pipelines into the network |

#### DATA SOURCES

| Source | File | Geographic scope |
|---|---|---|
| FNB-Gas | `Anlage2_Leitungsmeldungen_...xlsx` — pipeline submissions from potential operators | Germany |
| FNB-Gas | `Anlage3_FNB_Massnahmenliste_...xlsx` — FNB measure list (retrofit + new build) | Germany |
| ENTSO-G | `data/wasserstoff_kernnetz/h2_inframap_entsog.geojson` | Europe |

The two FNB-Gas files are downloaded automatically by Snakemake at runtime from fnb-gas.de. Both datasets are merged in `cluster_wasserstoff_kernnetz`. Duplicate entries between the two sources are removed to prevent double-counting of pipeline capacities.

#### PIPE CAPACITY CALCULATION

$H_2$ pipe capacity is estimated from pipe diameter using piecewise linear interpolation across three reference points:

**scripts/build_wasserstoff_kernnetz.py**

```
def diameter_to_capacity_h2(pipe_diameter_mm):
    """
    20 inch (500 mm)  50 bar ->  1.2 GW H2 (LHV)
    36 inch (900 mm)  50 bar ->  4.7 GW H2 (LHV)
    48 inch (1200 mm) 80 bar -> 16.9 GW H2 (LHV)
    """
    m0 = (1200 - 0) / (500 - 0)
    m1 = (4700 - 1200) / (900 - 500)
    m2 = (16900 - 4700) / (1200 - 900)

    if pipe_diameter_mm < 500:
        return m0 * pipe_diameter_mm
    elif pipe_diameter_mm < 900:
        return m1 * pipe_diameter_mm + (1200 - m1 * 500)
    else:
        return m2 * pipe_diameter_mm + (4700 - m2 * 900)
```

For retrofitted pipes, the displaced gas capacity is also tracked using `diameter_to_capacity` (from `build_gas_network.py`) to reduce the gas network capacity accordingly.

#### INTEGRATION INTO THE NETWORK

**PyPSA capacity parameters**

- `p_nom` — nominal (installed) capacity of a link in MW.
- `p_nom_extendable` — if `True`, the optimizer decides the capacity; if `False`, it is fixed at `p_nom`.
- `p_nom_min` / `p_nom_max` — lower and upper bounds on the capacity the optimizer can choose. `p_nom_max = p_nom` means no expansion beyond current size; `p_nom_min = 0` means the link can be fully decommissioned.

$H_2$ pipeline data is integrated into the network in the `modify_prenetwork` rule via `add_wasserstoff_kernnetz()`. Only pipelines with `build_year` within the current investment period are added. The function also:

- **Reduces gas pipeline capacity** (`p_nom`) on routes where kernnetz pipelines have been retrofitted from gas
- **Reduces $H_2$ retrofitting potential** (`p_nom_max` on $H_2$ pipeline retrofitted links) for all kernnetz pipelines built to date

**scripts/modify_prenetwork.py**

```
# Reduce gas pipeline p_nom for retrofitted kernnetz pipes
gas_pipes = n.links.query("carrier == 'gas pipeline'")
res_gas_pipes = reduce_capacity(gas_pipes, add_reversed_pipes(wkn_to_date), carrier="gas")
n.links.loc[n.links.carrier == "gas pipeline", "p_nom"] = res_gas_pipes["p_nom"]

# Reduce H2 retrofitting potential for all kernnetz pipes built to date
res_h2_pipes_retrofitted = reduce_capacity(
    h2_pipes_retrofitted,
    add_reversed_pipes(wkn),
    carrier="H2",
    target_attr="p_nom_max",
    conversion_rate=snakemake.config["sector"]["H2_retrofit_capacity_per_CH4"],
)
```

---

## How to Modify the Hydrogen Network

#### THE WASSERSTOFFKERNNETZ

The Wasserstoffkernnetz integration is controlled under `policy_plans` in the scenario config:

**config/scenarios/config.SE.yaml**

```
policy_plans:
  wasserstoff_kernnetz:
    enable: true
    optimize_after: 2040  # pipelines are non-extendable until this year
```

#### GAS-TO-$H_2$

Pipeline retrofitting is controlled separately under `sector:`:

**config/config.agora.yaml**

```yaml
sector:
  H2_retrofit: true
  H2_retrofit_capacity_per_CH4: 0.6  # capacity ratio H2/CH4
```

## 4.3.3 $CO_2$ Pipeline Network

### Overview

$CO_2$ pipeline transport is a standard feature of PyPSA-Eur v0.10.0, retained unchanged in PyPSA-IEI. The $CO_2$ pipeline and submarine pipeline investment costs are overridden from the technology-data baseline — see Costs for details.

When both `co2_spatial: true` and `co2network: true` are set, the model builds an endogenously expandable $CO_2$ transport network. Each pipeline is a **bidirectional PyPSA Link** connecting per-node $CO_2$ storage tanks, whose candidate routes are derived automatically from the existing electricity transmission topology. This allows the optimiser to decide where captured $CO_2$ is temporarily stored, transported between nodes, and ultimately sequestered underground — rather than assuming a single, spatially aggregated $CO_2$ pool.

---

### Snakemake Rule

**Rule:** `prepare_sector_network`
**Script:** `scripts/prepare_sector_network.py`

Two functions are called in sequence:

| Function | Purpose |
|---|---|
| `add_co2_tracking()` | Adds atmospheric and regional $CO_2$ buses, short-term storage tanks, sequestration stores, and optional vent links |
| `add_co2_network()` | Derives candidate pipeline routes from the electricity network and adds bidirectional pipeline links |

---

### Network Topology

Pipeline routes are derived automatically — no separate pipeline dataset is required. `create_network_topology()` scans all AC lines and DC links in the network, groups connections by unique node-pair, and averages their `length` and `underwater_fraction`:

**scripts/prepare_sector_network.py**

```
co2_links = create_network_topology(n, "CO2 pipeline ")
```

The resulting pipeline names follow the pattern `CO2 pipeline <nodeA> -> <nodeB>`. Connections where both ends belong to the same node are dropped automatically.

---

### Cost Calculation

Capital cost per pipeline route combines separate onshore and submarine cost rates weighted by the route's `underwater_fraction`:

**scripts/prepare_sector_network.py**

```
cost_onshore = (
    (1 - co2_links.underwater_fraction)
    * costs.at["CO2 pipeline", "fixed"]
    * co2_links.length
)
cost_submarine = (
    co2_links.underwater_fraction
    * costs.at["CO2 submarine pipeline", "fixed"]
    * co2_links.length
)
capital_cost = cost_onshore + cost_submarine
capital_cost *= snakemake.config["sector"]["co2_network_cost_factor"]
```

| Parameter | Source |
|---|---|
| `CO2 pipeline` fixed cost (€/MW/km/yr) | `data/costs_<year>.csv` — overridden, see Costs |
| `CO2 submarine pipeline` fixed cost (€/MW/km/yr) | `data/costs_<year>.csv` — overridden, see Costs |
| `CO2 pipeline` lifetime (years) | `data/costs_<year>.csv` |
| `co2_network_cost_factor` | `config.agora.yaml` → `sector:` |

Each pipeline is added as a fully bidirectional link (`p_min_pu=-1`), with capacity determined endogenously by the optimiser (`p_nom_extendable=True`):

**scripts/prepare_sector_network.py**

```
n.madd(
    "Link",
    co2_links.index,
    bus0=co2_links.bus0.values + " co2 stored",
    bus1=co2_links.bus1.values + " co2 stored",
    p_min_pu=-1,            # bidirectional: negative flow = reversed direction
    p_nom_extendable=True,
    length=co2_links.length.values,
    capital_cost=capital_cost.values,
    carrier="CO2 pipeline",
    lifetime=costs.at["CO2 pipeline", "lifetime"],
)
```

---

### Regional Sequestration Potential

Underground storage at `<node> co2 sequestered` is capped per node when `regional_co2_sequestration_potential.enable: true`. Geological potentials are sourced from the CO2Stop database (European Commission) and spatially overlaid with model regions in `scripts/`

`build_sequestration_potentials.py`. Per-node limits are clipped to `max_size` and divided by `years_of_storage` to convert a total geological capacity into an annual flow rate:

**scripts/prepare_sector_network.py**

```
e_nom_max = (
    e_nom_max
    .reindex(spatial.co2.locations)
    .fillna(0.0)
    .clip(upper=upper_limit)   # max_size in Mt
    .mul(1e6)                  # Mt → t
    / annualiser               # total capacity → annual rate
)
```

| Config key | Value | Meaning |
|---|---|---|
| `regional_co2_sequestration_potential.enable` | `true` | Apply per-node sequestration caps |
| `regional_co2_sequestration_potential.attribute` | `'conservative estimate Mt'` | Column from CO2Stop dataset to use |
| `regional_co2_sequestration_potential.include_onshore` | `true` | Include onshore geological formations |
| `regional_co2_sequestration_potential.min_size` | `3` | Minimum site size to include (Mt $CO_2$) |
| `regional_co2_sequestration_potential.max_size` | `25` | Hard upper cap per node (Mt $CO_2$) |
| `regional_co2_sequestration_potential.years_of_storage` | `25` | Annualisation divisor |
| `co2_sequestration_cost` | `10` | Cost (€/t $CO_2$) |
| `co2_sequestration_lifetime` | `50` | Lifetime (years) |

## Configuration

**config/config.agora.yaml**

```
sector:
  co2_spatial: true          # enable per-node CO₂ buses (required for CO₂ network)
  co2network: true           # add CO₂ pipeline links
  co2_network_cost_factor: 1 # cost multiplier (1 = unchanged)
  co2_vent: true             # allow venting of stored CO₂ back to atmosphere
  regional_co2_sequestration_potential:
    enable: true
    attribute: 'conservative estimate Mt'  # column from CO2Stop dataset
    include_onshore: true                  # include onshore formations
    min_size: 3                            # minimum site size (Mt CO₂)
    max_size: 25                           # upper cap per node (Mt CO₂)
    years_of_storage: 25                   # annualisation divisor
  co2_sequestration_potential: 500  # fallback global cap (Mt/yr) if regional disabled
  co2_sequestration_cost: 10        # €/t CO₂
  co2_sequestration_lifetime: 50    # years
```

## How to Modify the $CO_2$ Network

### DISABLE $CO_2$ PIPELINES WHILE KEEPING SPATIAL TRACKING

```
sector:
  co2_spatial: true
  co2network: false  # each node stores CO₂ independently, no transport
```

### DISABLE SPATIAL $CO_2$ TRACKING ENTIRELY

```
sector:
  co2_spatial: false  # all CO₂ collapses to a single EU-level bus
  co2network: false
```

### SCALE ALL PIPELINE COSTS

```
sector:
  co2_network_cost_factor: 1.5  # 50% more expensive pipelines
```

### REMOVE REGIONAL SEQUESTRATION CAPS

```
sector:
  regional_co2_sequestration_potential:
    enable: false
  co2_sequestration_potential: .inf  # unlimited global sequestration
```

## 4.3.4 TYNDP Electricity Projects

### Overview

TYNDP electricity transmission projects are enforced on the network in `prepare_sector_network`. Per-project capacity limits are read from two CSV files and applied for each planning horizon. Cross-border lines and links can optionally be made extendable from a given year onwards.

This covers **cross-border** TYNDP transmission projects. National (intra-country) transmission is handled separately via National Grid Plans.

---

### Snakemake Rule

**Rule:** `prepare_sector_network`
**Script:** `scripts/prepare_sector_network.py` — function `set_capacity_tyndp_elec()`

**Input data:**

- `data/links_2020-2050.csv` — DC link capacity limits per year
- `data/lines_2020-2050.csv` — AC line capacity additions per year and TYNDP status

---

### Scenario Behaviour

| Scenario | TYNDP statuses included | Cross-border extendable from |
|---|---|---|
| CE, CN | `under construction` | All horizons (`optimize_after: true`) |
| SE, SN | `in planning`, `in permitting`, `under construction` | 2040 onwards (`optimize_after: 2040`) |

---

### CSV Data Format

> **PyPSA capacity parameters**
>
> - `p_nom` — nominal (installed) capacity of a component in MW.
> - `p_nom_extendable` — if `True`, the optimizer decides the capacity; if `False`, it is fixed at `p_nom`.
> - `p_nom_min` / `p_nom_max` — lower and upper bounds on the capacity the optimizer can choose. Setting both equal to `p_nom` fixes the capacity; setting only `p_nom_min` leaves it extendable without an upper limit.

`data/links_2020-2050.csv`

Columns: `name`, `bus0`, `bus1`, `p_nom`, `2020`, `2025`, ..., `2050`

Each year column contains a fraction of `p_nom` or the special keyword `opt` (= set a soft minimum and leave the link extendable without an upper bound):

| Value | Effect |
|---|---|
| `0.0` | Link not active in this year |
| `1` (float > 0) | `p_nom_min = p_nom_max = p_nom × value`, link made extendable |
| `opt` | `p_nom_min = p_nom`, link remains extendable without upper bound |

**`data/lines_2020-2050.csv`**

Columns: `name`, `bus0`, `bus1`, then one column per `{year}{status}` combination (e.g., `2025under construction`, `2030in planning`).

Each cell holds the capacity addition [MW] for that year and status, or `opt` for a soft minimum (extendable without upper bound). Values with an `opt` suffix set the minimum and leave the line extendable.

---

### Configuration

TYNDP electricity enforcement is disabled by default and enabled per scenario:

**config/config.agora.yaml**

```yaml
policy_plans:
  include_tyndp_elec:
    enable: false           # set to true in scenario configs
    allowed_statuses:
      - under consideration
      - in planning
      - in permitting
      - under construction
    optimize_after: true    # true = all horizons; integer year = from that year
```

**config/scenarios/config.CE.yaml**

```yaml
policy_plans:
  include_tyndp_elec:
    enable: true
    allowed_statuses:
      - under construction
    # optimize_after inherits true from base config
```

**config/scenarios/config.SE.yaml**

```yaml
policy_plans:
  include_tyndp_elec:
    enable: true
    allowed_statuses:
      - in planning
      - in permitting
      - under construction
    optimize_after: 2040
```

---

### How to Change the TYNDP enforcement

#### CONFIGURE WHICH TYNDP PROJECTS ARE ENFORCED

Edit `policy_plans.include_tyndp_elec.allowed_statuses` in the scenario config. Only projects whose TYNDP status tag matches an entry in this list are enforced.

#### CONFIGURE WHEN CROSS-BORDER LINKS BECOME EXTENDABLE

Set `policy_plans.include_tyndp_elec.optimize_after` to `true` (all horizons) or to an integer year (e.g., `2040`) in the scenario config.

#### MODIFY CAPACITY VALUES PER HORIZON

Edit the year columns in `data/links_2020-2050.csv` (DC links) or the `{year}{status}` columns in `data/lines_2020-2050.csv` (AC lines).

## 4.3.5 National Grid Expansion Plans

### Overview

National electricity transmission expansion is constrained based on country-specific grid expansion plans (e.g., from Ember study). The feature limits national AC line and DC link expansion to specified factors relative to base year capacities.

This applies only to **national** (intra-country) transmission infrastructure. Cross-border TYNDP projects are handled separately via the TYNDP Electricity Projects functionality.

---

### Snakemake Rule

**Rule:** `solve_elec_networks` / `solve_sector_networks`
**Script:** `scripts/solve_network.py` — function `add_national_grid_plan_constraints()`

**Input data:**

- `data/national_line_expansions.csv` — Expansion factors per country and year

---

### Scenario Behaviour

The constraint is disabled by default and enabled per scenario via `solving.constraints.national_grid_plans: true`. The `optimize_after` parameter controls from which planning horizon national grids become freely extendable (no longer bound by the CSV expansion factors).

| Scenario | Constraint active | `optimize_after` | Effect |
|---|---|---|---|
| CE | No | — | No national expansion limit applied |
| CN | No | — | No national expansion limit applied |
| SE | Yes | `2040` | Expansion capped by CSV factors; freely extendable from 2040 |
| SN | Yes | `2040` | Expansion capped by CSV factors; freely extendable from 2040 |

**PyPSA capacity parameters**

PyPSA uses different capacity attributes depending on component type:

- `p_nom` — nominal power capacity of a **Link** (DC line, converter, etc.) in MW. `p_nom_opt` is the value chosen by the optimizer; `p_nom_min` / `p_nom_max` are the bounds.
- `s_nom` — nominal apparent power capacity of a **Line** (AC transmission) in MVA. `s_nom_opt` is the optimized value; `s_nom_min` / `s_nom_max` are the bounds.
- `s_max_pu` — per-unit loading limit on a Line (typically 0.7), so effective capacity = `s_nom × s_max_pu`.
- `p_nom_extendable` / `s_nom_extendable` — if `True`, the optimizer decides the capacity.

**Base year capacities:** The reference capacities that factors are applied to are derived from the optimized network of the **first planning horizon** (e.g. 2020). They are computed by `extract_base_year_capacities()` in `scripts/add_brownfield.py` when preparing the second planning horizon, summing national AC line ( `s_nom_opt × s_max_pu` ) and DC link ( `p_nom_opt` ) capacities per country, and saved to `results/.../base_year_capacities/`. The constraint then enforces `factor × base_capacity`.

**Factor selection per country:** For each country, the code searches the CSV for data years that fall in the window `(previous_horizon, current_horizon]`. The most recent year within that window is used as the expansion factor. Countries with no data in that window are excluded from the constraint.

**Fallback when no country has data in the window** (e.g. all countries lack CSV entries for this period):

| Case | Condition | Behavior |
|---|---|---|
| 1A — Before data range | Current year < first CSV year, and not the first planning horizon | National transmission **fixed** at current capacity (except TYNDP-committed projects) |
| 1B — First planning horizon | Current year is the first in `planning_horizons` | All national transmission **freely extendable** |
| 1C — Beyond data range | Current year > last CSV year (and not first horizon) | Controlled by `optimize_after` setting |

### CSV Data Format

`data/national_line_expansions.csv`

Columns: Country ISO2 code as row index, year columns (e.g., `2026`, `2030`, ...):

```
country,2026,2030
DK,1.44,
EE,1.17,1.24
GR,1.16,1.35
DE,1.09,1.2
FR,1.01,1.02
PL,1.12,1.24
LV,1.0,1.0
```

Each cell contains the **expansion factor** (float) relative to base year capacity:

| Value | Meaning |
|---|---|
| `1.0` | No expansion allowed beyond base year (e.g. LV) |
| `1.09` | 9% expansion allowed (e.g. DE in 2026) |
| `1.44` | 44% expansion allowed (e.g. DK in 2026) |
| Empty/NaN | Fallback behavior applies (e.g. DK in 2030) |

---

### Configuration

Default values in `config/config.agora.yaml` — the constraint is off by default:

**config/config.agora.yaml**

```yaml
policy_plans:
  include_national_grid_plans:
    national_grid_plan_data: data/national_line_expansions.csv
    national_expansion_condition: equal  # 'min', 'max', or 'equal'
    optimize_after: true  # true = freely extendable at all horizons (no cap once constraint is off)

solving:
  constraints:
    national_grid_plans: false  # disabled by default; enable per scenario
```

Overrides in SE and SN scenarios — these are the only two scenarios with the constraint active:

**config/scenarios/config.SE.yaml | config/scenarios/config.SN.yaml**

```yaml
policy_plans:
  include_national_grid_plans:
    optimize_after: 2040  # expansion capped by CSV factors until 2040, freely extendable from 2040

solving:
  constraints:
    national_grid_plans: true
```

| Parameter | Options | Description |
|---|---|---|
| `national_grid_plan_data` | File path | CSV with expansion factors per country/year |
| `national_expansion_condition` | `min`, `max`, `equal` | Constraint type: ≥, ≤, or = |
| `optimize_after` | `true` or integer | When national grids become freely extendable |
| `national_grid_plans` (in solving) | Boolean | Enable/disable constraint per scenario |

### Constraint type behavior:

- **`equal`** (default): National capacity must equal factor × base capacity
- **`min`**: National capacity at least factor × base capacity (allows more)
- **`max`**: National capacity at most factor × base capacity (caps expansion)

## How to Modify

#### CHANGE EXPANSION FACTORS PER COUNTRY/YEAR

The active expansion data file is set in the config under `policy_plans`:

**config/config.agora.yaml**

```yaml
policy_plans:
  include_national_grid_plans:
    national_grid_plan_data: data/national_line_expansions.csv
```

You have two options depending on whether the change should apply to **all runs** or only to **specific scenarios**.

### Option 1: Edit the shared file (applies to all runs)

Edit `data/national_line_expansions.csv` directly. All runs that reference this file will pick up the change.

### Option 2: Create a scenario-specific file (applies to selected runs only)

1. Copy the base file and give it a descriptive name:

```
cp data/national_line_expansions.csv data/national_line_expansions_constrained.csv
```

2. Edit the copy with the scenario-specific expansion factors.
3. Point to it in the relevant scenario config under `config/scenarios/`:

**config/scenarios/config.MY_SCENARIO.yaml**

```yaml
policy_plans:
  include_national_grid_plans:
    national_grid_plan_data: data/national_line_expansions_constrained.csv
```

This overrides only `national_grid_plan_data` for that scenario; all other config values remain inherited from `config.agora.yaml`.

> **Tip**
>
> This approach is useful when comparing scenarios with different grid ambition levels — e.g. a baseline with Ember factors and a sensitivity with tighter national expansion limits — without touching the shared file.

#### ADJUST CONSTRAINT TYPE

Set `national_expansion_condition` to `min`, `max`, or `equal` in config.

#### CONTROL WHEN NATIONAL GRIDS BECOME FREELY EXTENDABLE

Set `optimize_after` to `true` (always) or integer year (e.g., `2040`).

#### ENABLE/DISABLE PER SCENARIO

Set `solving.constraints.national_grid_plans: true` in scenario config.

---

**Interaction with TYNDP**

National grid plans apply **after** TYNDP electricity constraints. Lines/links already fixed by TYNDP (where `p_nom_min = p_nom_max` or `s_nom_min = s_nom_max`) are excluded from national grid constraints, ensuring TYNDP commitments take precedence.

**Data Source**

Default factors based on Ember transmission grids study.

## 4.3.6 Import Infrastructure Retrofitting

### Overview

Allows **retrofitting of existing natural gas import infrastructure** (both pipelines and LNG terminals) to hydrogen, and enables **dual use** of gas infrastructure by syngas. This constraint models the reuse of import capacity as energy carriers transition from fossil to renewable/synthetic fuels.

Two types of infrastructure can be retrofitted or shared:

1. **Pipeline imports**: $CH_4$ pipelines → $H_2$ pipelines
2. **LNG terminals**: Liquefied natural gas → Liquefied hydrogen

Additionally, natural gas infrastructure can be **dual-used** by syngas without retrofitting.

**Scenario Behavior:** This constraint is **always enabled for all scenarios** and applies automatically wherever import generators for multiple gas types exist at the same location. Unlike other constraints, it cannot be disabled via scenario configuration.

---

### Snakemake Rule

**Script:** `scripts/solve_network.py` — function `add_import_retrofit_constraint`, called from `extra_functionality`
**Rule:** `solve_sector_network_myopic`

---

### How It Works

#### PIPELINE RETROFITTING

For locations where both `import pipeline gas` and `import pipeline-H2` generators exist (same cluster), the following constraint applies:

**Constraint:**
$CH_4$ capacity + ($H_2$ capacity / retrofit factor) = Initial $CH_4$ capacity

Where:
- $CH_4$ capacity = optimized natural gas pipeline capacity
- $H_2$ capacity = optimized hydrogen pipeline capacity
- Retrofit factor = `H2_retrofit_capacity_per_CH4` (default: 0.6)
- Initial $CH_4$ capacity = existing natural gas pipeline capacity (`p_nom_max`)

**Interpretation:** 1 MW of $CH_4$ pipeline can transport 0.6 MW of $H_2$ after retrofitting (due to lower volumetric energy density). Total usage cannot exceed the original $CH_4$ capacity in energy-equivalent terms.

#### LNG TERMINAL RETROFITTING

For locations with both `import lng gas` and `import shipping-H2 (retrofitted)`:

**Constraint:**
LNG capacity + Liquefied $H_2$ capacity = Initial LNG capacity

LNG terminals can be split between natural gas and liquefied hydrogen imports, with total capacity constrained to the original LNG terminal size.

#### DUAL USE BY SYNGAS

Natural gas pipelines and LNG terminals can be **simultaneously used** by syngas without capacity conversion factors:

**Pipelines:**
At each time step: $CH_4$ dispatch + Syngas dispatch ≤ $CH_4$ installed capacity

**LNG terminals:**
At each time step: LNG dispatch + Syngas-shipping dispatch ≤ LNG installed capacity

Additionally, the installed capacity of syngas import generators is set equal to the natural gas capacity to enable dual use.

---

### Config

```yaml
sector:
  H2_retrofit_capacity_per_CH4: 0.6  # 1 MW CH4 → 0.6 MW H2
```

**`H2_retrofit_capacity_per_CH4`**: Ratio of hydrogen transport capacity to methane capacity after retrofitting a pipeline (unitless, typically 0.5–0.7 due to $H_2$'s lower volumetric energy density).

---

### How to Modify

#### CHANGE $H_2$ RETROFIT EFFICIENCY

```yaml
sector:
  H2_retrofit_capacity_per_CH4: 0.5  # more conservative (1 MW CH4 → 0.5 MW H2)
```

#### DISABLE RETROFITTING (NOT RECOMMENDED)

The constraint is automatically applied where import generators exist. To effectively disable:

1. Remove $H_2$ import generators from `data/import_nodes_tech_manipulated_s_62_<year>.csv`, or
2. Remove `add_import_retrofit_constraint(n)` call from `extra_functionality` in `solve_network.py` (modifies core code)

# 4.4 📈 Demand & Supply

## 4.4.1 Industrial Energy Demand

### Overview

Industry demand is updated from the standard PyPSA-Eur values using data from the TransHyDe project. Two modifications are made relative to base PyPSA-Eur:

1. **Carrier ratios** for high-temperature processes are remapped to hydrogen to better reflect intra-country spatial distribution
2. **Absolute country totals** are scaled to match TransHyDe demand values

---

### Snakemake Rules

| Rule | Script | Purpose |
|---|---|---|
| `build_industry_sector_ratios` | `scripts/build_industry_sector_ratios.py` | Set carrier ratios per industry sector |
| `build_industrial_energy_demand_per_node` | `scripts/build_industrial_energy_demand_per_node.py` | Scale absolute demands to TransHyDe totals |

---

### Demand Scenarios

Three demand scenarios are defined in `build_industry_sector_ratios.py`:

| `demand_scenario` | Description |
|---|---|
| `original` | Reverts to base PyPSA-Eur v0.10.0 carrier ratios |
| `high-temperature` | Hydrogen replaces fossil fuels in furnaces and smelters — reflects TransHyDe Scenario 1.5 (Mid Demand) |
| `high-temperature+steam` | As above, plus hydrogen replaces steam processing — reflects TransHyDe Scenario 2 (High Demand) |

The carrier ratio scenario is selected via `transhyde_processes` in the config:

**config/config.agora.yaml**

```yaml
industry:
  transhyde_data: true
  transhyde_processes: high_temp_only  # use "high-temperature" ratios
  transhyde_scenario: 1.5              # selects input CSV: TH_S1.5_demand.csv
```

`transhyde_scenario` ( `1.5` or `2` ) and `transhyde_processes` are **independent**: `transhyde_scenario` selects which TransHyDE CSV is loaded for absolute demand scaling; `transhyde_processes` controls carrier ratios. They can be combined freely — for example, the `CE_TranshydeS2` scenario uses `transhyde_scenario: 2` with the default `transhyde_processes: high_temp_only` .

---

### Hydrogen Demand Distribution

When `transhyde_data: true` , carrier ratios for high-temperature industrial processes (furnaces, smelters) are remapped to hydrogen. This only affects **spatial distribution** within a country — the absolute country totals are overwritten in the next step. From the script docstring:

> When `transhyde_data` is True, processes (mostly heat) are mapped from originally electricity/biomass/gas to hydrogen. The mapping is not precise; the aim is to get a more realistic distribution of the demand within a country.

---

### Scaling to TransHyDe Totals

`build_industrial_energy_demand_per_node` reads TransHyDe CSV data and scales PyPSA-Eur's country-level totals to match. The scaling preserves the intra-country distribution from `build_industry_sector_ratios` :

**scripts/build_industrial_energy_demand_per_node.py**

```python
# Calculate per-carrier scaling factor: TransHyDe total / PyPSA total
factor = (demand_th_p / nodal_df_grp_m).fillna(1.0)

# Apply factor to all nodes, preserving intra-country distribution
nodal_df *= factor

# Where PyPSA has zero demand but TransHyDe has non-zero,
# distribute TransHyDe total using node weights
```

The TransHyDe data files are in `data/transhyde/` .

---

### Ammonia and Methanol Imports

The model can reduce industrial energy demand when ammonia or methanol are imported rather than produced domestically. Both are disabled by default:

**config/config.agora.yaml**

```yaml
sector:
  ammonia: false
  ammonia_import: false
  ammonia_import_share: 50   # % of demand met by imports, if enabled
  methanol_import: false
  methanol_import_share: 50  # % of demand met by imports, if enabled
```

---

### How to Change the TransHyDe Scenario

Set `transhyde_processes` to control carrier ratios, and `transhyde_scenario` to select the input CSV for absolute scaling:

```yaml
industry:
  transhyde_processes: high_temp_only  # default; or: with_steam
  transhyde_scenario: 1.5              # default (Mid Demand); or: 2 (High Demand)
```

#### USING A CUSTOM DEMAND FILE

`transhyde_scenario` is used directly to build the input file paths:

```
data/transhyde/TH_S{transhyde_scenario}_demand.csv
data/transhyde/TH_S{transhyde_scenario}_process_emissions.csv
```

You can create a custom variant of either file and point to it by choosing any string as the scenario name. Both files must exist under the same name.

1. Copy both base files and give them a shared custom suffix:

```
cp data/transhyde/TH_S1.5_demand.csv            data/transhyde/TH_Scustom_demand.csv
cp data/transhyde/TH_S1.5_process_emissions.csv  data/transhyde/TH_Scustom_process_emissions.csv
```

2. Edit the demand or emissions file with the scenario-specific values.
3. Point to it in the relevant scenario config under `config/scenarios/`:

**config/scenarios/config.MY_SCENARIO.yaml**

```yaml
industry:
  transhyde_scenario: custom
```

Snakemake will then read `TH_Scustom_demand.csv` and `TH_Scustom_process_emissions.csv` for that scenario only.

**Tip**

If you only need to modify demand (not process emissions), still copy the emissions file unchanged — both must be present for the rule to run.

## 4.4.2 Energy Imports

### Overview

Energy imports are modelled as **extendable generators** (with supporting buses and links where needed), covering five import technologies: hydrogen (pipeline and shipping), syngas (pipeline and shipping), and synfuel. Import capacities, costs, and node assignments are loaded from a planning-year-specific CSV file derived from the TransHyDe project.

---

### Snakemake Rule

**Rule:** `prepare_sector_network`
**Script:** `scripts/prepare_sector_network.py` — function `add_import()`
**Data:** `data/import_nodes_tech_manipulated_s_62_<year>.csv`

---

### Import Technologies

| Technology | Components added | Carrier |
|---|---|---|
| `pipeline-H2` | Generator → `{node} H2` bus | `import pipeline-H2` |
| `shipping-H2` | Generator → `{node} H2` bus | `import shipping-H2` |
| `pipeline-syngas` | Bus ( `syngas` ) + Generator + Link ( `syngas→gas` ) | `import pipeline-syngas` |
| `shipping-syngas` | Generator → existing `syngas` bus + Link ( `syngas→gas` ) | `import shipping-syngas` |
| `synfuel` | Bus ( `synfuel` ) + Generator + Link ( `synfuel→EU oil` ) | `synfuel` |

For $H_2$ (pipeline and shipping), only a generator is needed since $H_2$ buses already exist. For syngas and synfuel, an intermediate bus is created and a link converts to the model's gas/oil network with a **negative $CO_2$ efficiency** on the `co2 atmosphere` bus — this cancels out the emissions that would otherwise be attributed when the synthetic fuel is eventually consumed.

---

### Implementation

**$H_2$ (pipeline and shipping)** — generator only, injecting directly into the node's existing $H_2$ bus. Shipping routes include `shipping_idx` in the component name to distinguish multiple routes to the same node:

**scripts/prepare_sector_network.py**

```
# pipeline-H2: generator → {node} H2
n.madd(
    "Generator",
    [f"{node} {tech}" for node, tech in zip(import_nodes_tech_manipulated_pipeline_h2["bus"],
```

```
                                        import_nodes_tech_manipulated_pipeline_h2["tech"])],
    bus=[f"{node} H2" for node in import_nodes_tech_manipulated_pipeline_h2["bus"]],
    carrier=[f"import {tech}" for tech in import_nodes_tech_manipulated_pipeline_h2["tech"]],
    p_nom_extendable=True,
    p_nom_max=[f"{p_nom}" for p_nom in import_nodes_tech_manipulated_pipeline_h2["p_nom"]],
    marginal_cost=[f"{marginal_cost}" for marginal_cost in import_nodes_tech_manipulated_pipeline_h2["marginal_cost"]],
    capital_cost=[f"{capital_cost}" for capital_cost in import_nodes_tech_manipulated_pipeline_h2["capital_cost"]],
    p_min_pu=0,
    lifetime=25,
)

# shipping-H2: same structure, shipping_idx appended to name
n.madd(
    "Generator",
    [f"{node} {tech} ({shipping_idx})" for node, tech, shipping_idx in zip(
        import_nodes_tech_manipulated_shipping_h2["bus"],
        import_nodes_tech_manipulated_shipping_h2["tech"],
        import_nodes_tech_manipulated_shipping_h2["shipping_idx"])],
    bus=[f"{node} H2" for node in import_nodes_tech_manipulated_shipping_h2["bus"]],
    carrier=[f"import {tech}" for tech in import_nodes_tech_manipulated_shipping_h2["tech"]],
    p_nom_extendable=True,
    p_nom_max=[f"{p_nom}" for p_nom in import_nodes_tech_manipulated_shipping_h2["p_nom"]],
    marginal_cost=[f"{marginal_cost}" for marginal_cost in import_nodes_tech_manipulated_shipping_h2["marginal_cost"]],
    capital_cost=[f"{capital_cost}" for capital_cost in import_nodes_tech_manipulated_shipping_h2["capital_cost"]],
    p_min_pu=0,
    lifetime=25,
)
```

**Syngas (pipeline and shipping)** — an intermediate `syngas` bus is created per node, then a link converts syngas → gas with `efficiency2=-CO2_intensity` on the `co2 atmosphere` bus:

**scripts/prepare_sector_network.py**

```
# syngas bus – created once for the union of pipeline-syngas and shipping-syngas nodes
n.madd(
    "Bus",
    [f"{node} syngas" for node in pd.concat([
        import_nodes_tech_manipulated_pipeline_syngas["bus"],
        import_nodes_tech_manipulated_shipping_syngas["bus"]
    ]).unique()],
    location=[f"{node}" for node in pd.concat([
        import_nodes_tech_manipulated_pipeline_syngas["bus"],
        import_nodes_tech_manipulated_shipping_syngas["bus"]
    ]).unique()],
    carrier="syngas",
    unit="MWh_LHV",
)
```

**Synfuel** — same pattern: `synfuel` bus → generator → link to `EU oil` with `efficiency2=-costs.at["oil", "CO2 intensity"]`.

---

### Input CSV Format

`data/import_nodes_tech_manipulated_s_62_<year>.csv` — one row per import node and technology:

```
,bus,Hydrogen Derivate,p_nom,note,tech,shipping_idx,marginal_cost,capital_cost
0,IT1 2,0,36721.2,Pipeline Italien: p_nom=existing gas_pipeline*0.6,pipeline-H2,,97,2688.0
1,BE1 0,,18782.8,,pipeline-syngas,,163,0.0
15,BE1 0,,20780.2,,shipping-syngas,,163,0.0
```

```
47,EU,,inf,,synfuel,,161,0.0
48,BE1 0,Ammonia,23000.0,Zeebrugge New Molecules development,shipping-H2,new,99,8421.6
```

| Column | Description |
|---|---|
| `bus` | Model cluster node (e.g. `BE1 0`, `DE1 1`) or `EU` for European aggregate |
| `Hydrogen Derivate` | Hydrogen carrier type (e.g. Ammonia, LH2, LOHC) |
| `p_nom` | Maximum import capacity [MW] |
| `note` | Human-readable description of the import project(s) at this node |
| `tech` | Import technology string |
| `shipping_idx` | Unique index for shipping routes |
| `marginal_cost` | Variable cost [EUR/MWh] |
| `capital_cost` | Annualised investment cost [EUR/MW/year] |

---

## How to Modify Imports

#### IMPORT CAPACITIES AND COSTS

To change import capacities or costs, edit the relevant year file:

```
data/import_nodes_tech_manipulated_s_62_2030.csv
data/import_nodes_tech_manipulated_s_62_2035.csv
...
```

#### IMPORT NODES

To add a new import node, add a row with the correct bus name, technology, and capacity. Bus names must match nodes in the clustered network (`s_62` = 62-node clustering). To disable all imports, remove or comment out the `add_import(n)` call in `prepare_sector_network.py`.

# 4.5 ⚖️ Solver Constraints

## 4.5.1 Self-Sufficiency Constraints

### Overview

Minimum and maximum self-sufficiency constraints can be enforced for energy carriers at the country, regional cluster, or EU-wide level. The constraint ensures that a specified fraction of total demand is met by domestic generation rather than imports.

**Self-sufficiency** is the ratio of local generation to total demand
(local generation + net imports). For example, 70% self-sufficiency means that 70% of demand is met by domestic sources and 30% by imports.

The constraint is enforced on **annual energy** (MWh), not capacity. It applies to each planning year independently and accounts for:

- Domestic generation from all relevant technologies
- Cross-border electricity transmission (AC lines and DC links)
- Pipeline flows (hydrogen, synfuel)
- External imports from non-European regions

---

### Snakemake Rule

**Script:** `scripts/solve_network.py` — function `add_self_sufficiency_constraints`, called from `extra_functionality`
**Rule:** `solve_sector_network_myopic`
**Data file:** `data/self_sufficiency_limits.csv` (configured via `policy_plans.self_sufficiency_limits`)

---

### Scenario Behavior

| Scenario | Enabled | Use case |
|---|---|---|
| CE, SE | No | Cross-border planning without self-sufficiency targets |
| CN, SN | Yes | National self-sufficiency targets enforced |

Enable via scenario config:

```yaml
solving:
  constraints:
    self_sufficiency: true
```

---

### Supported Carriers

The constraint independently tracks self-sufficiency for three energy carriers:

| Carrier | ID in CSV | Generation technologies | Import pathways |
|---|---|---|---|
| **Hydrogen** | `H2` | $H_2$ Electrolysis, SMR, SMR CC | $H_2$ pipelines, external imports |
| **Electricity** | `AC` | Solar, onwind, offwind, hydro, nuclear, fossil CHP, $H_2$ fuel cells | AC lines, DC links |
| **Synfuel** | `synfuel` | Fischer-Tropsch, biomass to liquid | (compared to oil demand) |

For **electricity**, generation from the following carriers is counted: - `offwind-ac`, `offwind-dc`, `onwind`, `solar`, `solar rooftop`, `ror` (run-of-river) - `hydro` (storage units) - Conventional plants: `coal`, `lignite`, `OCGT`, `CCGT`, `nuclear`, `oil`, `allam` - CHPs: gas and biomass variants (residential, services, urban central) - $H_2$ conversion: `H2 Fuel Cell`, `H2 turbine`

For **hydrogen**, generation includes: - `H2 Electrolysis` (electrolyzers) - `SMR`, `SMR CC` (steam methane reforming with/without carbon capture)

For **synfuel**, the constraint compares: - Synfuel generation: `Fischer-Tropsch`, `biomass to liquid` - Oil demand: land transport oil, shipping oil, aviation kerosene, residential/services oil boilers, industrial naphtha, agriculture machinery oil

---

### Constraint Levels

The function can apply constraints at **multiple levels**:

1. **Per-country** (ISO2 codes: `FR`, `DE`, `IT`, etc.)
   Cross-border flows between countries count as imports/exports.
2. **Per-cluster** (cluster IDs like `PL0 0`, `DE1 0`, etc.)
   Flows between clusters count as imports/exports, even within the same country.
3. **EU-wide** (`EU` identifier)
   Aggregates generation and trade for all EU27 countries as a single entity.

**Default configuration:** The provided `data/self_sufficiency_limits.csv` contains only **country-level** and **EU-wide** constraints. Cluster-level constraints are technically supported by the code but not configured in the default dataset.

---

### Minimum and Maximum Constraints

Both lower and upper bounds can be specified:

- **Minimum self-sufficiency** (`min` column): Enforces at least X% domestic generation
  Local generation must be greater than or equal to (Total demand × min value).
- **Maximum self-sufficiency** (`max` column): Prevents over-generation (exports above threshold)
  Local generation must be less than or equal to (Total demand × max value).

Leave `min` or `max` blank (or empty rows) for no constraint in that direction.

---

### CSV Format

**File:** `data/self_sufficiency_limits.csv`

| Column | Description |
|---|---|
| `country` | ISO2 country code (e.g. `DE`, `FR`), cluster ID (e.g. `PL0 0`), or `EU` for EU-wide |
| `carrier` | Carrier: `H2`, `AC`, or `synfuel` |
| `year` | Planning year (integer) |
| `min` | Minimum self-sufficiency as a fraction (e.g. `0.8` = 80%) — blank = no minimum |
| `max` | Maximum self-sufficiency as a fraction (e.g. `1.1` = 110%, allowing 10% net exports) — blank = no maximum |

**Default data** (excerpt from `data/self_sufficiency_limits.csv`):

```
country,carrier,year,min,max
DE,AC,2030,0.8,1.1
DE,AC,2050,1.0,1.1
DE,H2,2030,0.7,1.1
EU,AC,2030,0.8,1.1
EU,H2,2030,0.7,1.1
```

In this default configuration: - All countries must produce ≥80% of electricity demand in 2030 (increasing to 100% by 2050) - All countries must produce ≥70% of $H_2$ demand (constant 2025–2050) - Maximum self-sufficiency is 110% (allowing 10% net exports) - EU-wide aggregates have the same targets as individual countries

---

### Config

The active file is set in the base config:

**config/config.agora.yaml**

```yaml
policy_plans:
  self_sufficiency_limits: data/self_sufficiency_limits.csv
```

Enable the constraint in a scenario config:

```yaml
solving:
  constraints:
    self_sufficiency: true
```

You have two options for customising the limits depending on scope.

**Option 1: Edit the shared file (applies to all runs)**

Edit `data/self_sufficiency_limits.csv` directly. All scenarios that load this file will pick up the change.

**Option 2: Create a scenario-specific file (applies to selected runs only)**

1. Copy the base file and give it a descriptive name:

```
cp data/self_sufficiency_limits.csv data/self_sufficiency_limits_stricter_H2.csv
```

2. Edit the copy with the scenario-specific targets.
3. Point to it in the relevant scenario config under `config/scenarios/`:

**config/scenarios/config.MY_SCENARIO.yaml**

```yaml
policy_plans:
  self_sufficiency_limits: data/self_sufficiency_limits_stricter_H2.csv
```

This overrides only `self_sufficiency_limits` for that scenario; all other config values remain inherited from `config.agora.yaml`.

> **Tip**
>
> This approach is useful when comparing scenarios with different self-sufficiency ambition levels — e.g. a relaxed baseline and a strict national scenario — without touching the shared file.

## How to Modify

#### CHANGE EXISTING TARGETS

The default CSV contains uniform targets for all countries. To adjust, modify the `min` or `max` values for specific countries and years:

```
DE,H2,2030,0.8,1.1  # increase Germany's H₂ target from 0.7 to 0.8
FR,AC,2050,0.9,1.2  # relax France's electricity target and allow more exports
```

#### ADD SELF-SUFFICIENCY TARGET FOR A NEW COUNTRY

Add rows for a country not in the default data:

```
CH,H2,2030,0.6,1.0
CH,AC,2030,0.85,1.1
```

#### CHANGE EU-WIDE ELECTRICITY TARGET

```
EU,AC,2050,0.75,1.0
```

EU must generate 75–100% of electricity demand in 2050 (no net imports, limited net exports).

#### RELAX CONSTRAINT FOR A SPECIFIC YEAR

Remove the corresponding row or leave both `min` and `max` blank:

```
FR,H2,2035,,
```

#### APPLY CLUSTER-LEVEL CONSTRAINTS

For finer-grained regional policy (not in default config):

```
PL0 0,AC,2030,0.9,
PL1 0,AC,2030,0.5,
```

Note: Cluster IDs must match the network clustering (e.g., `s_62` = 62 clusters) and follow the format `{country_code}{cluster_num} {sub_id}` with a space.

---

## Notes

**Energy vs. Capacity**

Unlike the expansion limits which constrain installed **capacity** (MW), self-sufficiency constraints operate on annual **energy** (MWh) generation and demand.

**External imports**

External imports (from non-European regions) are represented by generators with carriers like `import pipeline-H2` and `import shipping-H2`. These count as imports for the purpose of self-sufficiency calculation.

**Interaction with other constraints**

Self-sufficiency constraints are independent of: - Expansion limits (capacity bounds per technology)
- National grid plans (transmission expansion factors)
- TYNDP projects (cross-border transmission projects)

All constraints are applied simultaneously during optimization.

## 4.5.2 Overall Minimum Capacities

### Overview

Enforces minimum total installed capacity per carrier **across all nodes** in the model. Unlike per-country expansion limits, this constraint sets a floor on the **aggregate capacity** for specific generation technologies across the entire system (e.g., all of Europe).

**Origin:** This constraint is from **PyPSA-Eur**, where it was originally called "BAU" (business-as-usual). It has been renamed to `overall_min_capacities` for clarity, though the config parameter remains `BAU_mincapacities`.

This is useful for business-as-usual scenarios or policy mandates that require a minimum deployment of certain technologies regardless of where they are located.

**Example:** Enforce at least 100 GW of OCGT capacity across all nodes —
the sum of OCGT capacity across all nodes must be ≥ 100,000 MW.

---

### Snakemake Rule

**Script:** `scripts/solve_network.py` — function `add_overall_min_capacities_constraints`, called from `extra_functionality`
**Rule:** `solve_sector_network_myopic`

---

### Scenario Behavior

By default, this constraint is **disabled** in all scenarios.

Enable via scenario config:

```
solving:
  constraints:
    overall_min_capacities: true
```

---

### How It Works

For each carrier specified in `electricity.BAU_mincapacities`:

1. Sum the extendable `p_nom` capacity across all generators with that carrier
2. Constrain the sum to be ≥ the specified minimum (in MW)

Only applies to **extendable generators**. Existing (non-extendable) capacity is excluded from the constraint.

---

### Config

Add to your config (typically in a scenario file):

```yaml
electricity:
  BAU_mincapacities:
    solar: 50000      # minimum 50 GW solar across all nodes
    onwind: 100000    # minimum 100 GW onshore wind
    OCGT: 20000       # minimum 20 GW OCGT
    offwind-ac: 0     # no minimum (can be omitted)
    offwind-dc: 0     # no minimum (can be omitted)

solving:
  constraints:
    overall_min_capacities: true
```

**Format:** Carrier name → minimum capacity in MW

---

### How to Modify

#### ADD MINIMUM CAPACITY FOR A TECHNOLOGY

Add an entry to `electricity.BAU_mincapacities`:

```yaml
electricity:
  BAU_mincapacities:
    nuclear: 50000  # enforce at least 50 GW nuclear across Europe
```

#### REMOVE A CONSTRAINT

Set to `0` or remove the carrier entry:

```yaml
electricity:
  BAU_mincapacities:
    solar: 0  # no minimum solar requirement
```

#### DISABLE THE CONSTRAINT ENTIRELY

```yaml
solving:
  constraints:
    overall_min_capacities: false
```

## 4.5.3 Capacity Reserve Margin

### Overview

Enforces a **capacity reserve margin** as a percentage above peak demand, requiring sufficient dispatchable generation capacity to cover peak load plus a safety margin. Only **conventional (dispatchable) generation** counts toward the reserve; renewable and storage units are excluded.

**Origin:** This constraint is from **PyPSA-Eur**, where it was originally called "SAFE" (capacity reserve). It has been renamed to `capacity_reserve` for clarity.

This constraint models security-of-supply requirements or capacity markets where a minimum firm capacity must be maintained regardless of renewables penetration.

**Example:** 10% reserve margin means total conventional capacity must be ≥110% of peak demand.

---

### Snakemake Rule

**Script:** `scripts/solve_network.py` — function `add_capacity_reserve_constraints`, called from `extra_functionality`
**Rule:** `solve_sector_network_myopic`

---

### Scenario Behavior

By default, this constraint is **disabled** in all scenarios.

Enable via scenario config:

```
solving:
  constraints:
    capacity_reserve: true
```

---

### How It Works

1. Calculate peak demand as the maximum total load across all time steps
2. Apply reserve margin: Required capacity = Peak demand × (1 + margin)
3. Sum existing + extendable conventional generation capacity
4. Constrain: Total conventional capacity ≥ Required capacity

**Conventional carriers** are defined in `electricity.conventional_carriers`: - `nuclear`, `coal`, `lignite`, `oil`, `OCGT`, `CCGT`, `geothermal`, `biomass`

**Excluded** from the constraint:
- Renewable generators (solar, wind)

- Storage units (batteries, hydro)
- Transmission capacity

---

### Config

Add to your config (typically in a scenario file):

```yaml
electricity:
  capacity_reserve_margin: 0.1  # 10% reserve above peak demand
  conventional_carriers:
    - nuclear
    - oil
    - OCGT
    - CCGT
    - coal
    - lignite
    - geothermal
    - biomass

solving:
  constraints:
    capacity_reserve: true
```

`capacity_reserve_margin`: Fraction above peak demand (0.1 = 10%, 0.15 = 15%, etc.)

---

### How to Modify

#### CHANGE THE RESERVE MARGIN

```yaml
electricity:
  capacity_reserve_margin: 0.15  # increase to 15%
```

#### DEFINE WHICH CARRIERS COUNT AS "CONVENTIONAL"

Edit the list to include/exclude specific technologies:

```yaml
electricity:
  conventional_carriers:
    - nuclear
    - CCGT  # exclude OCGT and coal from reserve requirement
```

#### DISABLE THE CONSTRAINT

```yaml
solving:
  constraints:
    capacity_reserve: false
```

## 4.5.4 Expansion Limits

### Overview

Minimum and maximum capacity constraints are enforced for selected renewable and other carriers across European countries for each planning year. The constraints target **total cumulative installed capacity** per country and carrier. Existing (non-extendable) capacity is subtracted before the bound is applied to the extendable `p_nom` decision variable.

---

### Snakemake Rule

**Script:** `scripts/solve_network.py` — function `add_country_carrier_limit_constraints`, called from `extra_functionality`
**Rule:** `solve_sector_network_myopic`
**Data files:** selected via `policy_plans.agg_p_nom_limits` in the scenario config

| Scenario | Data file | Min/max band |
|---|---|---|
| CE, SE, and variants | `data/agg_p_nom_minmax_european.csv` | ±20 % |
| CN, SN | `data/agg_p_nom_minmax_national.csv` | ±15 % |

---

### Carriers and Components

| Carrier | Component | Geographic scope | Years with data |
|---|---|---|---|
| `solar` | Generator | All modelled countries | 2030, 2050 |
| `onwind` | Generator | All modelled countries | 2030, 2050 |
| `offwind` | Generator | All modelled countries | 2030, 2050 |
| `H2 Electrolysis` | Link | European-wide ( `Eur` ) + Romania | 2030 |
| `gas for electricity` | Link | Poland (2020–2030), Romania (2020–2050) | varies |
| `co2 sequestered` | Store | Romania | 2030–2050 |

`gas for electricity` is a **grouped carrier**: OCGT, CCGT, urban central gas CHP (and CHP CC), micro gas CHP variants, and `allam` are all counted together against this limit. For Links, the constraint is applied to output capacity ( `p_nom × efficiency` ), not input capacity.

---

### CSV Format

Both files share the same long format: one row per (country, carrier, year) combination. Empty `min` or `max` cells mean no constraint for that direction.

| Column | Description |
|---|---|
| `country` | ISO2 country code, or `Eur` for European-wide aggregate |
| `carrier` | Carrier name as it appears in the constraint (see grouping above) |
| `year` | Planning year (integer) |
| `min` | Minimum total installed capacity [MW or t for $CO_2$] — blank = unconstrained |
| `max` | Maximum total installed capacity [MW or t for $CO_2$] — blank = unconstrained |
| `unit` | Unit string (e.g. `MW`, `MW_H2`, `t`) |
| `Source` | Source reference |
| `component` | PyPSA component type in **lowercase** (`generator`, `link`, `store`) |

Example rows:

```
country,carrier,year,min,max,unit,Source,component
DE,solar,2030,172000.0,258000.0,MW,Ember,generator
DE,solar,2050,9600.0,175423.9,MW,Atlite,generator
PL,gas for electricity,2025,,5500.0,MW,Forum Energii 2025/03/07,link
Eur,H2 Electrolysis,2030,46978.8,111532.4,MW_H2,IEA hydrogen map,link
RO,co2 sequestered,2030,,10000000.0,t,NZIA communicated by EPG,store
```

### Config

The active file is set via `policy_plans.agg_p_nom_limits`. The base config uses the European file; scenario configs override it:

**config/config.agora.yaml**

```yaml
policy_plans:
  agg_p_nom_limits: data/agg_p_nom_minmax_european.csv  # default
```

**config/scenarios/config.CN.yaml | config.SN.yaml**

```yaml
policy_plans:
  agg_p_nom_limits: data/agg_p_nom_minmax_national.csv
```

> **Note**
>
> The two files differ only in the ±% band applied to the 2030 Ember/NECP targets: ±20 % in the European file and ±15 % in the national file. The 2050 upper bounds (Atlite technical potentials) and all non-renewable entries are identical.

You have two options for customising the limits.

**Option 1: Edit a shared file (applies to all runs that use it)**

Edit `data/agg_p_nom_minmax_european.csv` or `data/agg_p_nom_minmax_national.csv` directly. All scenarios that reference the file will pick up the change.

**Option 2: Create a scenario-specific file (applies to selected runs only)**

1. Copy the relevant base file and give it a descriptive name:

```
cp data/agg_p_nom_minmax_european.csv data/agg_p_nom_minmax_custom.csv
```

2. Edit the copy with the scenario-specific capacity limits.
3. Point to it in the relevant scenario config under `config/scenarios/`:

**config/scenarios/config.MY_SCENARIO.yaml**

```yaml
policy_plans:
  agg_p_nom_limits: data/agg_p_nom_minmax_custom.csv
```

This overrides only `agg_p_nom_limits` for that scenario; all other config values remain inherited from `config.agora.yaml`.

> **Tip**
>
> This is useful when comparing scenarios with different renewable expansion ambition levels — e.g. a moderate baseline and a high-ambition sensitivity — without modifying the shared files.

---

## How to Modify the Expansion Limits

#### CHANGE A CAPACITY LIMIT FOR A COUNTRY

Edit the relevant CSV directly. Each row is a single (country, carrier, year) entry. For example, to raise the solar minimum in Spain for 2030:

```
ES,solar,2030,130000.0,195000.0,MW,Custom,generator
```

#### ADD A CONSTRAINT TO NEW COUNTRY OR CARRIER

Add a new row with `country`, `carrier`, `year`, `min` / `max`, and `component`. Make sure `carrier` matches the grouped name (e.g. use `offwind`, not `offwind-ac`) and `component` is lowercase.

#### ADD A EUROPEAN-WIDE CONSTRAINT

Use `Eur` as the `country` value. The code automatically duplicates every component into an `Eur` group and adds both country-level and European-level `p_nom` to the left-hand side of the constraint.

# 4.6 🎯 Optimization Settings

## 4.6.1 Myopic Optimization (Brownfield)

### Overview

Myopic pathway optimization builds each planning horizon on top of the previous one. The `add_brownfield` rule manages this transition by:

1. Carrying forward previously built capacity (fixed, non-extendable)
2. Accounting for gas pipeline retrofitting to $H_2$
3. Adding newly commissioned powerplants
4. Updating import pipeline and terminal retrofit potentials

---

### Snakemake Rule

**Rule:** `add_brownfield`
**Script:** `scripts/add_brownfield.py` — function `add_brownfield()`

---

### 1. Carrying Forward Previous Capacity

All assets (Generator, Link, Store) from the previous horizon that are still within their lifetime are imported into the current network with their **optimized capacity fixed** (non-extendable):

**scripts/add_brownfield.py**

```
for c in n_p.iterate_components(["Link", "Generator", "Store"]):
    # Remove assets that have expired
    n_p.mremove(c.name, c.df.index[c.df.build_year + c.df.lifetime < year])

    # Remove assets below minimum capacity threshold
    n_p.mremove(c.name, c.df.index[
        c.df[f"{attr}_nom_extendable"] & (c.df[f"{attr}_nom_opt"] < threshold)
    ])

    # Fix capacity at optimized value from previous horizon
    c.df[f"{attr}_nom"] = c.df[f"{attr}_nom_opt"]
    c.df[f"{attr}_nom_extendable"] = False

    n.import_components_from_dataframe(c.df, c.name)
```

---

### 2. Gas Pipeline Retrofitting

When `H2_retrofit: true`, existing gas pipeline capacity is reduced by the amount already retrofitted to $H_2$ in previous horizons:

**scripts/add_brownfield.py**

```
CH4_per_H2 = 1 / snakemake.params.H2_retrofit_capacity_per_CH4

# Already retrofitted H2 capacity from previous years
already_retrofitted = (
    n.links.loc[h2_retrofitted_fixed_i, "p_nom"]
    .rename(lambda x: x.split("-2")[0].replace(fr, to) + f"-{year}")
    .groupby(level=0).sum()
)

# Remaining gas capacity = original - retrofitted (adjusted for energy ratio)
remaining_capacity = (
    pipe_capacity - CH4_per_H2
    * already_retrofitted.reindex(pipe_capacity.index).fillna(0)
)
n.links.loc[gas_pipes_i, "p_nom"] = remaining_capacity
n.links.loc[gas_pipes_i, "p_nom_max"] = remaining_capacity
```

---

### 3. New Powerplants

A base powerplant network is loaded and newly commissioned plants are added for the current horizon:

**scripts/add_brownfield.py**

```
n_ppl = pypsa.Network(snakemake.input.base_powerplants)
all_years = snakemake.config["scenario"]["planning_horizons"]
previous_year = str(all_years[all_years.index(year) - 1])

for ppl_component in n_ppl.iterate_components(["Link", "Generator"]):
    # Select plants commissioned between previous and current year
    idx_ppl = ppl_component.df.index[
        ends_with_year(ppl_component.df.index)
        & (ppl_component.df.index.str[-4:] > previous_year)
        & (ppl_component.df.index.str[-4:] <= str(year))
    ]
    n.generators.loc[idx_ppl, 'p_nom'] = ppl_component.df.loc[idx_ppl, 'p_nom']
```

---

### 4. Import Pipeline and Terminal Retrofitting

The same retrofit logic is applied to non-European gas import pipelines and LNG terminals:

**scripts/add_brownfield.py**

```
# Remaining gas pipeline capacity after retrofit
remaining_capacity_pipes = (
    full_capacity_pipes
    - CH4_per_H2 * retrofitted_capacity_pipes
      .rename(lambda x: x[:5] + ' import pipeline gas')
      .groupby(level=0).sum()
)
n.generators.loc[filtered_natural_gas_pipes_i, 'p_nom_max'] = remaining_capacity_pipes

# Remaining H2 retrofit potential
remaining_capacity_pipes_retro = (
```

```
    max_capacity_retrofit_pipes
    - retrofitted_capacity_pipes
      .rename(lambda x: x[:-4] + str(year))
      .groupby(level=0).sum()
)
n.generators.loc[filtered_h2_pipes_ext_i, 'p_nom_max'] = remaining_capacity_pipes_retro
```

The same pattern applies to LNG → $H_2$ terminal retrofitting.

---

### How to Configure

**config/config.agora.yaml**

```
sector:
  H2_retrofit: true                     # enable gas-to-H2 retrofitting
  H2_retrofit_capacity_per_CH4: 0.6     # H2 capacity per CH4 capacity unit

existing_capacities:
  threshold_capacity: 10                # MW — assets below this are dropped
```

Planning horizons are defined in the scenario config:

**config/scenarios/config.CE.yaml**

```
scenario:
  planning_horizons: [2020, 2025, 2030, 2035, 2040, 2045, 2050]
```

## 4.6.2 Curtailment Mode

### Overview

An optional solver feature that explicitly models renewable curtailment by adding virtual generators with negative costs. When enabled, all renewable generators are forced to track their maximum availability ( `p_min_pu = p_max_pu` ) and negative-cost virtual generators absorb excess electricity that cannot be consumed or stored.

This makes curtailment volumes directly visible in the optimization results, useful for analyzing renewable integration and grid flexibility constraints.

The feature is **disabled by default** and can be enabled in the config file.

---

### Snakemake Rule

**Rule:** `solve_elec_networks` / `solve_sector_networks`
**Script:** `scripts/solve_network.py` — function `prepare_network()`

---

### Implementation

> **PyPSA per-unit dispatch parameters**
>
> - `p_max_pu` — upper bound on dispatch in each time step, as a fraction of `p_nom` (e.g. `0.8` = the generator can produce at most 80% of its capacity in that hour). For renewables this is the availability profile.
> - `p_min_pu` — lower bound on dispatch, also as a fraction of `p_nom`. Setting `p_min_pu = p_max_pu` forces the generator to run at exactly its available capacity (no curtailment allowed unless a curtailment generator absorbs the excess).
>
> These are **time-varying dispatch bounds**, distinct from `p_nom_min` / `p_nom_max` which are bounds on the *installed* capacity chosen by the optimizer.

When enabled, curtailment is modeled explicitly rather than implicitly:

1. All renewable generators have their `p_min_pu` set equal to `p_max_pu` (forcing them to "commit" to their full available capacity)
2. Virtual curtailment generators are added at all AC buses with:
3. Negative marginal cost (-0.1 EUR/MWh) to incentivize curtailment
4. Negative dispatch range ( `p_min_pu = -1` , `p_max_pu = 0` )
5. Large nominal capacity (1,000 GW)

This allows the optimizer to explicitly curtail renewables when grid constraints or storage limitations prevent full utilization, making curtailment volumes directly visible in the results.

CODE

**scripts/solve_network.py**

```
if solve_opts.get("curtailment_mode"):
    n.add("Carrier", "curtailment", color="#fedfed", nice_name="Curtailment")
    n.generators_t.p_min_pu = n.generators_t.p_max_pu
    buses_i = n.buses.query("carrier == 'AC'").index
    n.madd(
        "Generator",
        buses_i,
        suffix=" curtailment",
        bus=buses_i,
        p_min_pu=-1,
        p_max_pu=0,
        marginal_cost=-0.1,
        carrier="curtailment",
        p_nom=1e6,
    )
```

### Configuration

The feature is controlled under `solving.options`:

**config/config.agora.yaml**

```
solving:
  options:
    curtailment_mode: false   # boolean
```

### How to Enable

To enable curtailment mode for a specific scenario:

1. **Edit config file** — Set `curtailment_mode: true` in `config/config.agora.yaml` under `solving.options`, or
2. **Override in scenario config** — Add the option to a scenario-specific YAML file if using scenario overrides

The feature applies to all planning years in the optimization run. Curtailment results will appear in the network outputs under the `curtailment` carrier and can be analyzed via:

- `n.generators_t.p` (negative values for curtailment generators)
- `n.statistics()` with carrier filtering
- Post-processing scripts that aggregate by carrier

# 5. 📊 Post-Analysis

## 5.1 Post-Analysis Overview

The post-analysis scripts are located in `scripts_analysis/` and are run **after** the PyPSA optimization is complete. The master script `analysis_main.py` orchestrates all analyses and produces plots and Excel files organized into timestamped output folders.

### 5.1.1 How to Run

```
cd scripts_analysis
python analysis_main.py
```

The script must be run from inside `scripts_analysis/` — `main_dir` is resolved automatically as the parent of the current working directory.

### 5.1.2 Output Structure

```
analysis_results_YYYYMMDD/
  ExogenousDemand/      ← transport demand plots
  Balances/             ← energy balance charts per carrier
  network_plots/        ← transmission network maps
  KPIs/                 ← installed capacity, GWkm, dispatch, summary KPIs
  self-sufficiency/     ← regional energy self-sufficiency
  costs/                ← total system cost breakdown
  utilization/          ← transmission line utilization
  Import/               ← energy import volumes and costs
  balance_maps/         ← spatial energy balance maps
  marginal_prices/      ← nodal marginal prices per carrier
  h2_generation/        ← H₂ production volumes and full-load hours
  grid_emission_factors/ ← hourly CO₂ emission factors per AC node
```

## 5.1.3 Analysis Modules

| Output folder | Script | Function |
|---|---|---|
| `ExogenousDemand/` | `exogenous_demand_analyses.py` | `get_transport_demand_plot()` |
| `Balances/` | `configurable_energy_balances.py` | `get_standard_balances()` |
| `network_plots/` | `plot_elec_network.py` | `plot_map_elec_years()` |
| `network_plots/` | `plot_h2_network.py` | `plot_h2_map_years()` |
| `network_plots/` | `plot_co2_network.py` | `plot_co2_map_years()` |
| `network_plots/` | `plot_ch4_network.py` | `plot_ch4_map_years()` |
| `network_plots/` | `compare_grids.py` | `plot_grid_comparisons()` |
| `KPIs/` | `installed_capacity.py` | `call_installed_capacity_plot()` |
| `KPIs/` | `plot_GWkm.py` | `plot_TWkm_all_carriers()` |
| `KPIs/` | `summary_KPIs.py` | `start_KPI_analysis()` |
| `KPIs/` | `plot_cases_KPIs.py` | `plot_case_study_KPIs()` |
| `KPIs/` | `plot_dispatch_barchart.py` | `plot_dispatch_barchart()` |
| `self-sufficiency/` | `regional_self_sufficiency_level.py` | `evaluate_self_sufficiency()` |
| `costs/` | `analyze_total_system_cost.py` | `analyze_system_cost()` |
| `utilization/` | `line_usage.py` | `evaluate_line_usage()` |
| `Import/` | `import_analysis.py` | `analyze_imports()` |
| `balance_maps/` | `plot_balance_map.py` | `plot_balance_map_years()` |
| `marginal_prices/` | `marginal_prices.py` | `get_marginal_prices()` |
| `h2_generation/` | `h2_generation.py` | `analyze_h2_generation()` |
| `grid_emission_factors/` | `grid_emission_factors.py` | `analyze_grid_emission_factors()` |

## 5.1.4 Country Scopes

Three country lists are defined in `analysis_main.py`:

| Variable | Count | Countries | Used for |
|---|---|---|---|
| `countries` | 33 | EU + NO, GB, CH, Western Balkans | Full model scope |
| `eu27_countries` | 25 | EU27 excl. Malta, Cyprus | Network maps |
| `th_countries` | 26 | TransHyDE project countries | Import analysis |

# 5.2 System Costs

## 5.2.1 Overview

Total system cost breakdown per scenario and planning year, covering capital and operational expenditure by technology group. Costs are shown both as annual values and as integrated cumulative costs from the start year to each planning horizon.

**Script:** `scripts_analysis/analyze_total_system_cost.py`
**Function:** `analyze_system_cost()`
**Output:** `costs/`

## 5.2.2 Output Files

| File | Description |
|---|---|
| `infrastructure_capital_cost.png` | Capital costs of transmission infrastructure (electricity, $H_2$, gas, $CO_2$) |
| `absolute_capital_cost.png` | Absolute capital costs by technology group across all scenarios |
| `residual_capital_cost_{sel_scen}_{scenario}.png` | Difference in capital costs vs `sel_scen`, one file per comparison scenario |
| `operational_costs.png` | Operational (marginal) costs by category across all scenarios and years |
| `residual_operational_costs_{sel_scen}_{scenario}.png` | Difference in operational costs vs `sel_scen`, one file per comparison scenario |
| `total_costs.png` | Total annualised costs (capex + opex) per scenario and year |
| `total_overinvestment.png` | System cost difference vs `sel_scen` (bn EUR/yr) |
| `integrated_total_costs.png` | Cumulative total costs from start year per scenario |
| `integrated_difference_costs.png` | Cumulative cost difference vs `sel_scen` (bn EUR) |
| `total_costs.xlsx` | Excel with annualised and integrated costs by category |

**sel_scen**

`sel_scen` is the primary scenario selected in `analysis_main.py`. It is used as the reference baseline for all residual and difference plots. Files containing `{sel_scen}` and `{scenario}` in their name are generated once per comparison scenario.

The Excel file is also automatically written into `KPIs/Summary-Results-KPIs.xlsx` if that file already exists.

## 5.2.3 Technology Mapping

Cost components are mapped to groups via:

```
scripts_analysis/Mapping_system_costs.csv
```

The groups used in plots are:

| Group | Includes |
|---|---|
| `infrastructure` | AC/DC lines, $H_2$/gas/$CO_2$ pipelines |
| `power generation` | Solar, wind, nuclear, hydro, etc. |
| `heat generation` | Heat pumps, boilers, CHPs |
| `transformation` | Electrolysers, methanisation, Fischer-Tropsch, etc. |
| `import (capex)` | Import infrastructure capital costs |
| `import (opex)` | Import fuel operational costs |
| `operational costs` | Fuel costs, VOM |
| `others (capex/opex)` | Remaining components |

## 5.2.4 Backup Capacities

`backup_capas` is hardcoded in `analysis_main.py` (currently `True`). When `True` and the backup cost file exists, gas turbine backup capacity costs are added to `total_overinvestment.png` and `integrated_difference_costs.png`. If the file is missing, the flag is automatically set to `False` by the script.

The costs are read from:

```
{analysis_results_dir}/Balances/gas_turbine_backup_costs.xlsx
```

where `{analysis_results_dir}` is the dated output folder (e.g., `analysis_results_20260311/`).

> **Note**
>
> This file is produced by `get_standard_balances()` for the `gas` carrier. The Energy Balances analysis must run before System Costs when `backup_capas=True`.

# 5.3 Energy Balances

## 5.3.1 Overview

Supply and demand balance charts are produced for 6 carriers across three regions (all countries, EU27, and one configurable country) per scenario and year. Results are saved as PNG bar charts, interactive HTML timeseries, and Excel files.

**Script:** `scripts_analysis/configurable_energy_balances.py`
**Function:** `get_standard_balances()`
**Output:** `Balances/`

## 5.3.2 Carriers

| Carrier | Description |
|---|---|
| `electricity` | High-voltage electricity |
| `gas` | Natural gas / methane |
| `low voltage` | Residential electricity distribution |
| `H2` | Hydrogen |
| `heat` | Decentral heat |
| `urban central heat` | District heating |

## 5.3.3 Output Files

For each carrier, the following are produced under `Balances/`:

| File / Folder | Description |
|---|---|
| `balance_{carrier}.xlsx` | Excel with supply/demand data per country and scenario |
| `gas_turbine_backup_capacities.xlsx` | Gas turbine backup capacity data (electricity carrier only) |
| `gas_turbine_backup_costs.xlsx` | Gas turbine backup cost data (electricity carrier only) |
| `bar_plots/` | PNG bar charts per region and scenario mode |
| `timeseries_plots_line_chart/` | Interactive HTML line charts (selectable scenario/year) |
| `timeseries_plots_stacked_bar_chart/` | Interactive HTML stacked bar chart + PNG per scenario/year |

Bar plots are generated for three regions: **All** (33 countries), **EU27**, and the configured country (default `DE`).

---

## 5.3.4 Configuration

### Country

`country_to_plot` is set in `analysis_main.py` (currently `"DE"`). Any ISO2 country code in the model scope can be used.

### Scenario comparison mode

`compare_scenarios` controls whether all scenarios are overlaid in one plot (`True`) or plotted individually (`False`). In `analysis_main.py` it is set to `[True, False]` for all carriers, producing both plot styles. `urban central heat` only uses `[True]` because it is not meaningful to plot it per individual scenario.

### PNG time-series resampling

PNG stacked area charts in `timeseries_plots_stacked_bar_chart/` show the January dispatch profile. By default the plots use **flexible time segments** from the model (e.g. 2190 segments for a segmented run), which renders very fast. Optionally the data can be resampled to hourly resolution first, producing a uniform x-axis — but this is significantly slower.

The behaviour is controlled by a module-level flag in `configurable_energy_balances.py`:

```
RESAMPLE_TIMESERIES_PNG = True  # default: uniform hourly x-axis
```

Override it once at the top of `analysis_main.py` before any balances are computed:

```
ceb.RESAMPLE_TIMESERIES_PNG = False  # fast: raw time segments
```

The current default in `analysis_main.py` is `False` (raw segments).

# 5.4 KPIs

## 5.4.1 Overview

Five functions produce key performance indicators covering installed capacity, infrastructure volume, dispatch, and summary system metrics.

## 5.4.2 Installed Capacity

**Script:** `installed_capacity.py`
**Function:** `call_installed_capacity_plot()`
**Output:** `KPIs/`

Installed capacity by technology, planning year, and scenario. Controlled by `plot_backup_capas` (boolean) in `analysis_main.py` to optionally include backup capacity.

Output files are produced for two scopes (`EU27` and `All`) and three capacity types (`generation`, `heat`, `storage`):

| File | Description |
|---|---|
| `installed_{type}_capacities_{scope}.png` | All-scenario comparison chart |
| `installed_{type}_capacities_{scope}_{scenario}.png` | Per-scenario chart across years |

## 5.4.3 Infrastructure Volume (TW·km)

**Script:** `plot_GWkm.py` **Function:** `plot_TWkm_all_carriers()` **Output:** `KPIs/`

Installed capacity × line length (TW·km) per carrier (electricity, $H_2$, $CO_2$, gas) across years and scenarios. Plots are produced for four regions: all modelled countries, EU27, Benelux, and the Baltic cluster.

| File | Description |
|---|---|
| `TWkm_elec_{region}.png` | Electricity TW·km |
| `TWkm_H2_{region}.png` | Hydrogen TW·km |
| `TWkm_CO2_{region}.png` | $CO_2$ TW·km |
| `TWkm_gas_{region}.png` | Gas TW·km |

Regions: `europe`, `eu27`, `benelux`, `balticum`.

## 5.4.4 Summary KPIs

**Script:** `summary_KPIs.py` **Function:** `start_KPI_analysis()` **Output:** `KPIs/`

Comprehensive Excel workbook covering total costs, emissions, and system metrics per country, EU27, and selected nodes, across all scenarios and years.

| File | Description |
|---|---|
| `Summary-Results-KPIs.xlsx` | Full KPI workbook |

---

## 5.4.5 Case Study KPIs

**Script:** `plot_cases_KPIs.py` **Function:** `plot_case_study_KPIs()` **Output:** `KPIs/`

Bar charts (with cluster map insets) comparing offshore capacity, $H_2$ storage, $CO_2$ storage, and $H_2$ import across scenarios and years for two predefined regional clusters: `northsea` and `balticum`.

| File | Description |
|---|---|
| `offshore_capacity_{cluster}.png` | Offshore installed capacity |
| `H2_storage_{cluster}.png` | $H_2$ storage capacity |
| `CO2_storage_{cluster}.png` | $CO_2$ storage capacity |
| `H2_import_{cluster}.png` | $H_2$ import (North Sea cluster only) |

---

## 5.4.6 Dispatch Barchart

**Script:** `plot_dispatch_barchart.py` — `plot_dispatch_barchart()`

Stacked bar chart of flexibility dispatch (storage, demand response, etc.) across scenarios and years. The `"flexibility"` argument selects the KPI type computed inside the script.

| File | Description |
|---|---|
| `flexibility_dispatch.png` | Flexibility dispatch comparison |

# 5.5 Exogenous Demand

## 5.5.1 Overview

Analysis of exogenous transport energy demand across planning years for a selected scenario. Transport demand is broken down by mode (aviation, shipping, land transport) and energy carrier (kerosene, oil, methanol, EV, fuel cell).

Results show total annual energy consumption (TWh) for each transport category, helping track the transition from fossil fuels to electrification and alternative fuels.

**Script:** `scripts_analysis/exogenous_demand_analyses.py`
**Function:** `get_transport_demand_plot()`
**Output:** `ExogenousDemand/`

## 5.5.2 Transport Categories

| Mode | Carrier | Description |
|---|---|---|
| Aviation | Kerosene | Kerosene and sustainable aviation fuel (SAF) |
| Shipping | Oil | Conventional marine fuel oil |
| Shipping | Methanol | Methanol-based marine fuel |
| Land transport | Oil | Conventional fossil fuel vehicles |
| Land transport | EV | Battery electric vehicles |
| Land transport | Fuel cell | Hydrogen fuel cell vehicles |

## 5.5.3 Output Files

| File | Description |
|---|---|
| `Transport_demand_exogenous.png` | Stacked bar chart showing transport demand by mode and carrier across years |
| `Transport_demand_exogenous.xlsx` | Excel data with annual energy demand (TWh) per transport category |

# 5.6 Import Analysis

## 5.6.1 Overview

Energy import volumes (TWh) per import technology across planning years and scenarios.

**Script:** `scripts_analysis/import_analysis.py`
**Function:** `analyze_imports()`
**Output:** `Import/`

---

## 5.6.2 Import Sets

Three sets of import carriers are evaluated independently:

| Set | Carriers |
|---|---|
| `H2_import` | $H_2$ pipeline, $H_2$ shipping |
| `H2_derivate_import` | $H_2$ pipeline, $H_2$ shipping, syngas pipeline, syngas shipping, synfuel |
| `energy_import` | All of the above + coal, oil, uranium, LNG gas, pipeline gas |

---

## 5.6.3 Output Files

For each import set the following files are saved to `Import/`:

| File | Description |
|---|---|
| `plot_{scenario}_{set}.png` | Stacked bar chart per scenario across years |
| `plot_all_{set}.png` | Side-by-side comparison of all scenarios |
| `imports_{scenario}_TWh.xlsx` | Detailed time-series data per scenario |

# 5.7 Self-Sufficiency

## 5.7.1 Overview

Regional energy self-sufficiency levels per country and planning year, showing the ratio of domestic generation to total demand. Analysis is performed for both hydrogen and electricity carriers across all EU27 countries plus surrounding regions.

The analysis is run pairwise: `sel_scen` is compared against each other scenario to produce difference maps.

**Script:** `scripts_analysis/regional_self_sufficiency_level.py`
**Function:** `evaluate_self_sufficiency()`
**Output:** `self-sufficiency/`

---

## 5.7.2 Carriers

Self-sufficiency is calculated independently for two carriers:

| Carrier | Description |
|---|---|
| `H2` | Hydrogen self-sufficiency based on domestic generation vs demand |
| `Electricity` | Electricity self-sufficiency based on renewable generation vs demand |

For each carrier, self-sufficiency accounts for: - Domestic generation (electrolysis for $H_2$, renewables for electricity) - Cross-border transmission (pipelines for $H_2$, AC/DC lines for electricity) - External imports (from outside Europe)

---

## 5.7.3 Output Files

For each scenario, year, and carrier, the following files are generated:

| File | Description |
|---|---|
| `{carrier}_self-sufficiency_map_{scenario}_{year}.png` | Map showing self-sufficiency percentage per country |
| `{carrier}_self-sufficiency_{scenario}_{year}.png` | Bar chart comparing self-sufficiency across countries, with EU27 aggregate |
| `{carrier}_self-sufficiency_diff_{scenario1}_{scenario2}_{year}.png` | Difference map comparing two scenarios |

Where `{carrier}` is either `h2` or `elec`.

**sel_scen**

`sel_scen` is the primary scenario selected in `analysis_main.py`. Difference maps are produced comparing `sel_scen` against each other scenario.

## 5.7.4 Regions

Self-sufficiency is calculated for:

- All 25 EU27 countries (excl. Malta and Cyprus)
- EU27 aggregate
- Surrounding countries (Balkans, UK, Norway, Switzerland)

The bar charts include an exogenous minimum self-sufficiency constraint line for scenarios `CN` and `SN` (years after 2025).

# 5.8 Balance Maps

## 5.8.1 Overview

Spatial maps of energy balances overlaid on the European grid, produced for each carrier, scenario, and year. A second set of difference maps compares the selected reference scenario against all other scenarios.

**Script:** `scripts_analysis/plot_balance_map.py`
**Function:** `plot_balance_map_years()`
**Output:** `balance_maps/`

## 5.8.2 Carriers

Carriers are defined in `config/config.plotting.yaml` under `constants.kinds`:

| Carrier | Description |
|---|---|
| `carbon` | Carbon flows ($CO_2$ capture, stored and sequestered) |
| `co2` | $CO_2$ emissions |
| `electricity` | Transmission level electricity (AC and DC) |
| `gas` | Natural gas / methane |
| `hydrogen` | Hydrogen |

## 5.8.3 Output Files

For each carrier, scenario, and year one PNG is saved to `balance_maps/`:

| File | Description |
|---|---|
| `{carrier}_{region}_balance_map_{scenario}_{year}.png` | Single-scenario balance map |
| `{carrier}_{region}_balance_map_{scen1} vs {scen2}_{year}.png` | What is additional in `scen1` relative to `scen2` |
| `{carrier}_{region}_balance_map_{scen2} vs {scen1}_{year}.png` | What is additional in `scen2` relative to `scen1` |

## 5.8.4 Configuration

### Region

`country_to_plot_networks` in `analysis_main.py` controls which region is plotted. It is set to `"EU27"` by default. Three regions have dedicated plot configurations in `config.plotting.yaml`:

| Value | Description |
|---|---|
| `"EU27"` | EU-27 + neighbouring countries (default) |
| `"PL_and_Baltics"` | Poland and the Baltic states (not plotted) |
| `"Benelux"` | Belgium, Netherlands, Luxembourg (not plotted) |

Any other value falls back to the `"EU27"` configuration.

### Reference scenario

`sel_scen` is the reference scenario used for the difference maps. Each other scenario in `scenarios` is compared against it.

> **Note**
>
> `sel_scen` is set in `analysis_main.py` and shared with other analysis functions (system costs, self-sufficiency).

# 5.9 Network Maps

## 5.9.1 Overview

Four scripts produce per-year and per-scenario maps of European transmission infrastructure. A fifth script compares two scenarios side by side.

**Output:** `network_plots/`

## 5.9.2 Scripts

| Script | Function | Network |
|---|---|---|
| `plot_elec_network.py` | `plot_map_elec_years()` | Electricity (AC lines + DC links) |
| `plot_h2_network.py` | `plot_h2_map_years()` | Hydrogen pipelines and storage |
| `plot_co2_network.py` | `plot_co2_map_years()` | $CO_2$ pipelines |
| `plot_ch4_network.py` | `plot_ch4_map_years()` | Methane / gas pipelines |
| `compare_grids.py` | `plot_grid_comparisons()` | Scenario difference maps |

## 5.9.3 Output Files

### Electricity

| File | Description |
|---|---|
| `elec_netexpansion_{scenario}_{year}.png` | Post-network + pre-network overlay |
| `pre_elec_netexpansion_{scenario}_{year}.png` | Net expansion (post − pre) |

### Hydrogen

| File | Description |
|---|---|
| `h2_electrolysis_network_{scenario}_{year}.png` | Electrolysis capacity map |
| `h2_all_network_{scenario}_{year}.png` | Full $H_2$ network |
| `h2_net_flows_network_{scenario}_{year}.png` | Net $H_2$ flows |
| `h2_storage_{scenario}_{year}.png` | $H_2$ storage capacity |

### $CO_2$ and Gas

| File | Description |
|---|---|
| `co2_network_{scenario}_{year}.png` | $CO_2$ pipeline network |
| `ch4_network_{scenario}_{year}.png` | Methane / gas pipeline network |

### Grid Comparisons

`plot_grid_comparisons()` runs for each scenario paired against `sel_scen`, producing electricity and $H_2$ difference maps per year:

| File | Description |
|---|---|
| `diff_elec_{scen1}_{scen2}_{year}.png` | Electricity grid difference |
| `diff_H2_{scen1}_{scen2}_{year}.png` | $H_2$ grid difference |

---

## 5.9.4 Geographic Scope

All network maps use `country_to_plot_networks`, set to `"EU27"` in `analysis_main.py`.

# 5.10 Gas Pipeline Utilization

## 5.10.1 Overview

Gas pipeline utilization analysis showing how heavily $CH_4$ pipelines are used relative to their nominal capacity. Utilization rates are calculated as mean power flow divided by capacity across all timesteps, weighted by snapshot duration.

Maps visualize both capacity (linewidth) and utilization (color scale) for each corridor across planning years and scenarios.

**Script:** `scripts_analysis/line_usage.py`
**Function:** `evaluate_line_usage()`
**Output:** `utilization/`

## 5.10.2 Output Files

For each scenario and year combination:

| File | Description |
|---|---|
| `ch4_utilization_map_{scenario}_{year}.png` | Map showing gas pipeline capacity (linewidth) and average utilization percentage (color) |

## 5.10.3 Visualization Details

**Pipeline Capacity (Linewidth)** - Linewidth scaled by optimal pipeline capacity ( `p_nom_opt` ) - Pipelines below 100 MW threshold are not displayed - Bidirectional flows are aggregated

**Utilization Rate (Color)** - Color scale from red (low utilization) to green (high utilization) - Calculated as: mean absolute power flow / (capacity × availability) - Averaged across the full year weighted by snapshot duration

# 5.11 Marginal Prices

## 5.11.1 Overview

Nodal marginal prices are extracted from the optimised networks for selected bus carriers across all scenarios and planning years. Results are written to Excel files, one per carrier.

**Script:** `scripts_analysis/marginal_prices.py`
**Function:** `get_marginal_prices()`
**Output:** `marginal_prices/`

---

## 5.11.2 Carriers

| Carrier | Bus type | Output label |
|---|---|---|
| `AC` | High-voltage electricity buses | `electricity` |
| `H2` | Hydrogen buses | `hydrogen` |

The default carrier list is `["AC", "H2"]`. It can be changed by passing a custom `carriers` argument to `get_marginal_prices()`.

---

## 5.11.3 Output Files

| File | Description |
|---|---|
| `marginal_prices_electricity.xlsx` | Nodal electricity marginal prices, one sheet per bus |
| `marginal_prices_hydrogen.xlsx` | Nodal hydrogen marginal prices, one sheet per bus |

Each Excel file contains one sheet per bus node. Within each sheet, rows are time steps and columns form a `MultiIndex(scenario, year)`.

---

## 5.11.4 How It Works

`get_marginal_prices()` calls two internal functions:

**`extract_marginal_prices()`**

Reads `network.buses_t.marginal_price` for each `(scenario, year)` pair and accumulates one narrow DataFrame per bus. Missing buses in a given network become `NaN` rather than raising an error.

```
mp = n.buses_t.marginal_price.reindex(columns=buses)
```

The result is a nested dict `{carrier: {bus: DataFrame}}` where each DataFrame has `MultiIndex(scenario, year)` columns and time steps as the index.

**`write_marginal_prices()`**

Writes the per-bus DataFrames to Excel, with each bus as a separate sheet:

```
for bus, df in prices[carrier].items():
    df.to_excel(writer, sheet_name=str(bus))
```

---

## 5.11.5 Configuration

`get_marginal_prices()` is called from `analysis_main.py`. The carriers and result directory are controlled there:

```
get_marginal_prices(
    networks_year=networks,
    years=years,
    scenarios=scenarios,
    scenario_colors=scenario_colors,
    resultdir=marginal_prices_dir,
    carriers=["AC", "H2"],  # default
)
```

# 5.12 Hydrogen Generation

## 5.12.1 Overview

Annual $H_2$ production volumes and full-load hours are extracted from the optimised networks per technology, bus node, scenario, and planning year. Results are written to a single Excel file with one sheet per $H_2$ bus.

**Script:** `scripts_analysis/h2_generation.py`
**Function:** `analyze_h2_generation()`
**Output:** `h2_generation/`

## 5.12.2 Technologies

| Technology | Description |
|---|---|
| `H2 Electrolysis` | Water electrolysis driven by electricity |
| `SMR` | Steam methane reforming (without carbon capture) |
| `SMR CC` | Steam methane reforming with carbon capture |

## 5.12.3 Output Files

| File | Description |
|---|---|
| `hydrogen_generation.xlsx` | $H_2$ generation and full-load hours per node, one sheet per $H_2$ bus |

Each sheet corresponds to one $H_2$ bus node (suffix `H2` stripped from the sheet name). Rows follow this order:

| Row group | Rows |
|---|---|
| $H_2$ generation [GWh_H2] | `H2 Electrolysis`, `SMR`, `SMR CC`, `Total` |
| Full-load hours [h/a] | `H2 Electrolysis`, `SMR`, `SMR CC` |

Columns form a `MultiIndex(scenario, year)`.

## 5.12.4 How It Works

`analyze_h2_generation()` calls two internal functions:

#### `extract_h2_metrics()`

Uses `n.statistics.energy_balance()` to obtain annually-weighted $H_2$ generation for each technology at each $H_2$ bus (supply side only, positive values):

```
eb = n.statistics.energy_balance(aggregate_bus=False)
s = eb.loc[pd.IndexSlice[:, carrier, :]]
s = s[s.index.get_level_values(-1).isin(h2_buses)]
s = s[s > 0].groupby(level=-1).sum()
```

Full-load hours per technology are computed as annual generation divided by optimised $H_2$ output capacity (taking into account link efficiency):

```
cap = (links.p_nom_opt * links.efficiency).groupby(links["bus1"]).sum()
flh = (gen / cap.replace(0, float("nan"))).round(1)
```

The result is a DataFrame with `MultiIndex(metric, technology, node)` as the index and `MultiIndex(scenario, year)` as columns.

#### `write_h2_metrics()`

Writes the metrics to Excel, one sheet per $H_2$ bus node, with rows reindexed to the canonical display order:

```
row_order = [H2 generation per technology, Total, FLH per technology]
node_df.to_excel(writer, sheet_name=str(node).removesuffix(" H2"))
```

---

## 5.12.5 Configuration

`analyze_h2_generation()` is called from `analysis_main.py`:

```
analyze_h2_generation(
    networks=networks,
    years=years,
    scenarios=scenarios,
    resultdir=h2_gen_dir,
)
```

# 5.13 Grid Emission Factors

## 5.13.1 Overview

Hourly average grid emission factors [$tCO_2$/MWh_el] are computed per AC bus node for all scenarios and planning years. The factor at each node reflects the generation-weighted average $CO_2$ intensity of local electricity production at each time step.

**Script:** `scripts_analysis/grid_emission_factors.py`
**Function:** `analyze_grid_emission_factors()`
**Output:** `grid_emission_factors/`

---

## 5.13.2 Output Files

| File | Description |
|---|---|
| `grid_emission_factors.xlsx` | Hourly emission factors per AC node, one sheet per bus |

Each sheet corresponds to one AC bus node, sorted alphabetically. Rows are hourly time steps and columns form a `MultiIndex(scenario, year)`. Values are in $tCO_2$/MWh_el. Hours with no local generation yield `NaN`.

---

## 5.13.3 Emission Factor Definition

The emission factor at node $b$ at time $t$ is the generation-weighted average $CO_2$ intensity across all local generators:

$$ EF_b(t) = \frac{\sum_g p_g(t) \cdot \alpha_g}{\sum_g p_g(t)} $$

where $p_g(t)$ is the dispatch [MW] and $\alpha_g$ is the $CO_2$ emission intensity [$tCO_2$/MWh_el] of generator $g$'s carrier. Hours with zero total local generation yield `NaN`.

---

## 5.13.4 How It Works

`analyze_grid_emission_factors()` calls two internal functions:

`extract_grid_emission_factors()`

Three component types contribute to both the electricity and $CO_2$ numerator:

| Component | Electricity | $CO_2$ |
|---|---|---|
| **Generators** | `n.generators_t.p` | `n.carriers.co2_emissions` × dispatch |
| **Links** (local generation only) | AC output port | $CO_2$-atmosphere port |
| **StorageUnits** | `n.storage_units_t.p` clipped to ≥ 0 | 0 (no direct emissions) |

**Link filtering:** Links whose `bus0` is an AC bus (e.g. HVDC, back-to-back converters) are excluded — these are transmission, not local generation. Only links with an AC output port that do not start from an AC bus are included.

**$CO_2$ port detection:** The script identifies the single port per link that connects to a `co2` atmosphere bus. CCS plants' "$CO_2$ stored" ports have a different carrier and are therefore never matched.

Values near zero ($< 1\times10^{-4}$) are set to zero to suppress numerical noise.

`write_grid_emission_factors()`

Writes one sheet per AC bus to a single Excel file, sorted alphabetically by bus name:

```
for bus, df in sorted(emission_factors.items()):
    df.to_excel(writer, sheet_name=str(bus))
```

## 5.13.5 Configuration

`analyze_grid_emission_factors()` is called from `analysis_main.py`:

```
analyze_grid_emission_factors(
    networks_year=networks,
    years=years,
    scenarios=scenarios,
    resultdir=emission_factors_dir,
)
```

# 6. Release Notes

- Add missing data files to repository PR #36
- Add OETC toggle config option PR #39
- Add documentation on constraints and national grid expansion plan PR #17
- Add documentation on post-analysis scripts PR #8
- Fix path and add logging to post-analysis scripts PR #6
- Enable download of documentation as PDF using `mkdocs-to-pdf` and enable documentation preview in PRs PR #14
- Deploy documentation and add documentation preview PR #7
- Add documentation PR #5
- Apply `snakemake` fix to prevent "another PID running" gurobi issue PR #3
- Integrate OETC PR #1

# 7. Support & Reporting Issues

If you encounter a problem — whether during installation, while running the pipeline, or with unexpected results — please open an issue on the GitHub repository so the team can help.

**Issues tracker:** github.com/open-energy-transition/PyPSA-IEI/issues

For a general guide on how to create a GitHub issue, see the official GitHub documentation.

---

## 7.1 When to Open an Issue

Open an issue for any of the following:

- **Installation failure** — conda environment creation errors, missing packages, or import errors
- **Pipeline crash** — a Snakemake rule fails with an error traceback
- **Wrong or unexpected results** — outputs that differ from the published study without an obvious explanation
- **Missing or incorrect input data** — files that cannot be retrieved or produce errors when parsed
- **Documentation problems** — unclear instructions, broken links, or missing information

---

## 7.2 What to Include in Your Report

A good issue report allows the team to reproduce and fix the problem quickly. Please include the following:

**1. Environment information**

```
conda activate pypsa-iei
conda list | grep -E "pypsa|snakemake|python|linopy|gurobi"
```

Also state your operating system (Windows / Linux / macOS) and version.

**2. The error message** *(recommended)*

If possible, copy the complete traceback from the terminal as text — this allows the team to search and analyse it directly. If that is not convenient, a screenshot of the terminal output is also acceptable.

For Snakemake errors, the most useful information is usually in the log file for the failing rule rather than the terminal itself:

```
cat logs/<rule_name>.log
```

**3. The config file used**

State which scenario config was used (e.g. `config/scenarios/config.SE.yaml`) and paste any relevant settings you modified.

**4. Steps to reproduce** *(optional — helpful for unexpected results)*

If the issue is not a straightforward crash, briefly describe what you did and what you expected to happen instead. For example, which planning horizon failed, or which config settings were changed.

**5. Expected vs actual behaviour**

Briefly state what you expected to happen and what actually happened.

---

## 7.3 Checking Existing Issues

Before opening a new issue, search the existing issues to see if your problem has already been reported or resolved.